



---

## École Nationale Supérieure d'Arts et Métiers — Meknès

*Université Moulay Ismaïl*

Filière : Intelligence Artificielle et Technologies des données – Systèmes Industriels

---

*Rapport de Projet — Maintenance Industrielle et Prédictive*

# PrognoSense

Plateforme Intelligente de Maintenance Prédictive  
basée sur l'Analyse Vibratoire et l'Intelligence  
Artificielle

---

**Réalisé par :**

Souleymane DIALLO

**Encadré par :**

Pr. Zaki SMAIL

---

Année universitaire 2025–2026

# Résumé

La maintenance représente aujourd'hui un poste de coût majeur de l'industrie, et les arrêts non planifiés en constituent la part la plus pénalisante. Ce projet présente **PrognSense**, une plateforme logicielle complète de **maintenance prédictive industrielle** qui transforme l'analyse des signaux vibratoires en décisions de maintenance anticipées et chiffrées.

La plateforme met en œuvre l'intégralité de la chaîne de surveillance conditionnelle : acquisition de signaux (rééchantillonnage à 12,8 kHz, fenêtrage glissant), extraction d'indicateurs temporels et fréquentiels (RMS, kurtosis, facteur de crête, énergies par bande, entropie spectrale), diagnostic spectral des défauts (fréquences caractéristiques de roulements BPFO/BPFI/BSF/FTF, analyse d'enveloppe par transformée de Hilbert), évaluation de la sévérité selon la norme **ISO 10816/20816** (vitesse vibratoire en mm/s, zones A/B/C/D), détection d'anomalies non supervisée par auto-encodeur, classification des défauts et pronostic de durée de vie restante (RUL).

Cinq modèles d'apprentissage automatique (Decision Tree, Random Forest, XGBoost, MLP, Huber) ont été entraînés et comparés sur **quatre jeux de données de référence** totalisant plus de **66 000 fenêtres** : VBL-VA001 (100 % d'exactitude), CWRU (97,6 % avec XGBoost sur 10 classes), Mechanical Faults (89,6 % avec le MLP) et CMAPSS pour la régression RUL (MAE de **9,61 cycles**,  $R^2 = 0,90$  avec XGBoost). La méthodologie d'évaluation repose sur un découpage *par fichier source* interdisant toute fuite de données.

Au-delà du diagnostic, PrognSense intègre une couche d'**industrialisation et de confiance** : connecteurs terrain (OPC-UA, MQTT) alimentant une couche d'ingestion universelle, apprentissage du comportement sain *propre à chaque machine*, indicateurs de fiabilité *mesurés* (MTBF, MTTR, disponibilité, ROI), création automatique d'ordres de travail transmis à la GMAO, journal d'audit horodaté, versioning des modèles avec retour arrière, mesure du taux réel de fausses alarmes, et un **copilot** à base de modèle de langage ancré sur une base de connaissances normative (RAG). L'architecture est conçue pour passer à l'échelle (base time-series TimescaleDB).

**Mots-clés** : maintenance prédictive, analyse vibratoire, ISO 10816, détection d'anomalies, auto-encodeur, RUL, CMAPSS, CWRU, XGBoost, OPC-UA, GMAO, explicabilité, MLOps, FastAPI, React.

---

## Abstract

Maintenance is a major cost center in industry, and unplanned downtime is its most penalizing component. This project introduces **PrognSense**, a comprehensive software platform for **industrial predictive maintenance** that turns vibration signal analysis into anticipated, quantified maintenance decisions.

The platform implements the full condition-based maintenance pipeline : signal acquisition (resampling to 12.8 kHz, sliding windows), extraction of time- and frequency-domain indicators (RMS, kurtosis, crest factor, band energies, spectral entropy), spectral fault diagnosis

(bearing characteristic frequencies BPFO/BPFI/BSF/FTF, Hilbert envelope analysis), severity assessment per the **ISO 10816/20816** standard (vibration velocity in mm/s, zones A/B/C/D), unsupervised anomaly detection via an autoencoder, fault classification and Remaining Useful Life (RUL) prognosis.

Five machine-learning models were trained and benchmarked on **four reference datasets** totalling more than **66,000 windows** : VBL-VA001 (100 % accuracy), CWRU (97.6 % with XGBoost over 10 classes), Mechanical Faults (89.6 % with the MLP) and CMAPSS for RUL regression (MAE of **9.61 cycles**,  $R^2 = 0.90$  with XGBoost), using a leakage-free split by source file.

Beyond diagnosis, PrognSense adds an **industrialization and trust** layer : field connectors (OPC-UA, MQTT) feeding a universal ingestion endpoint, per-machine healthy baseline learning, *measured* reliability indicators (MTBF, MTTR, availability, ROI), automatic work-order generation pushed to the CMMS, a timestamped audit trail, model versioning with rollback, real false-alarm-rate measurement, and an LLM-based **copilot** grounded on a normative knowledge base (RAG). The architecture is designed to scale on a TimescaleDB time-series backend.

**Keywords** : predictive maintenance, vibration analysis, ISO 10816, anomaly detection, autoencoder, RUL, CMAPSS, CWRU, XGBoost, OPC-UA, CMMS, explainability, MLOps.

# Table des matières

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Résumé</b>                                                               | <b>i</b>  |
| <b>1 Introduction</b>                                                       | <b>1</b>  |
| 1.1 Contexte industriel . . . . .                                           | 1         |
| 1.2 Problématique . . . . .                                                 | 1         |
| 1.3 Objectifs du projet . . . . .                                           | 2         |
| 1.4 Démarche et organisation du rapport . . . . .                           | 2         |
| <b>2 État de l’art et positionnement</b>                                    | <b>3</b>  |
| 2.1 La maintenance conditionnelle et l’architecture OSA-CBM . . . . .       | 3         |
| 2.2 Techniques d’analyse vibratoire . . . . .                               | 3         |
| 2.3 Apprentissage automatique pour le diagnostic et le pronostic . . . . .  | 4         |
| 2.4 Normes applicables . . . . .                                            | 4         |
| 2.5 Solutions du marché et positionnement de PrognSense . . . . .           | 4         |
| <b>3 Analyse des jeux de données</b>                                        | <b>5</b>  |
| 3.1 Vue comparative . . . . .                                               | 5         |
| 3.2 VBL-VA001 — bancs d’essai vibratoires . . . . .                         | 5         |
| 3.3 CWRU — défauts de roulement (Case Western Reserve University) . . . . . | 5         |
| 3.4 Mechanical Faults — machine tournante multi-capteurs . . . . .          | 6         |
| 3.5 CMAPSS FD001 — pronostic de durée de vie (NASA) . . . . .               | 6         |
| 3.6 MCC5-THU — défauts d’engrenage . . . . .                                | 6         |
| 3.7 Pipeline d’unification . . . . .                                        | 6         |
| 3.8 Couverture des modes de défaillance : un spectre complet . . . . .      | 7         |
| <b>4 Architecture générale de la plateforme</b>                             | <b>8</b>  |
| 4.1 Vue d’ensemble . . . . .                                                | 8         |
| 4.2 Principes de conception . . . . .                                       | 8         |
| 4.3 Communication temps réel . . . . .                                      | 9         |
| 4.4 Technologies retenues et justification . . . . .                        | 9         |
| <b>5 Pipeline de traitement du signal et extraction des indicateurs</b>     | <b>11</b> |
| 5.1 Prétraitement . . . . .                                                 | 11        |
| 5.2 Indicateurs temporels . . . . .                                         | 11        |
| 5.3 Indicateurs fréquentiels . . . . .                                      | 11        |
| 5.4 Fréquences caractéristiques de roulement . . . . .                      | 12        |
| 5.5 Analyse d’enveloppe (démodulation de Hilbert) . . . . .                 | 12        |
| 5.6 Sévérité ISO 10816 : de l’accélération à la vitesse . . . . .           | 12        |
| 5.7 Indice de santé (Health Index) . . . . .                                | 13        |

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| <b>6</b>  | <b>Ingénierie des indicateurs (feature engineering)</b>           | <b>14</b> |
| 6.1       | Features vibratoires standard . . . . .                           | 14        |
| 6.2       | Features par cycles (CMAPSS) . . . . .                            | 14        |
| 6.3       | Features spécialisées engrenages . . . . .                        | 14        |
| 6.4       | Features multi-capteurs (Mechanical Faults) . . . . .             | 15        |
| 6.5       | Cache et reproductibilité . . . . .                               | 15        |
| <b>7</b>  | <b>Modèles d'apprentissage et benchmark</b>                       | <b>16</b> |
| 7.1       | Modèles évalués . . . . .                                         | 16        |
| 7.2       | Méthodologie d'évaluation . . . . .                               | 16        |
| 7.3       | Résultats de classification . . . . .                             | 17        |
| 7.3.1     | Analyse par dataset . . . . .                                     | 17        |
| 7.4       | Résultats de régression (RUL) . . . . .                           | 17        |
| 7.5       | Cas particulier : CNN 1D pour les engrenages (MCC5-THU) . . . . . | 18        |
| 7.6       | Sélection des modèles actifs . . . . .                            | 19        |
| <b>8</b>  | <b>Détection d'anomalies non supervisée</b>                       | <b>21</b> |
| 8.1       | Motivation . . . . .                                              | 21        |
| 8.2       | Auto-encodeur . . . . .                                           | 21        |
| 8.2.1     | Architecture . . . . .                                            | 21        |
| 8.2.2     | Principe de détection . . . . .                                   | 21        |
| 8.3       | Ensemble de détecteurs (vote par consensus) . . . . .             | 22        |
| 8.4       | Apprentissage du baseline propre à chaque machine . . . . .       | 22        |
| <b>9</b>  | <b>Architecture backend (FastAPI)</b>                             | <b>24</b> |
| 9.1       | Organisation . . . . .                                            | 24        |
| 9.2       | Le registre de modèles . . . . .                                  | 24        |
| 9.3       | La simulation temps réel . . . . .                                | 25        |
| 9.4       | Sécurité et persistance . . . . .                                 | 25        |
| 9.5       | Validation des entrées . . . . .                                  | 25        |
| <b>10</b> | <b>Interface utilisateur (React)</b>                              | <b>26</b> |
| 10.1      | Principes d'interface . . . . .                                   | 26        |
| 10.2      | Les pages fonctionnelles . . . . .                                | 26        |
| 10.2.1    | Connexion . . . . .                                               | 26        |
| 10.2.2    | Vue Globale . . . . .                                             | 26        |
| 10.2.3    | Monitoring . . . . .                                              | 26        |
| 10.2.4    | IA Lab . . . . .                                                  | 26        |
| 10.2.5    | Pronostic & RUL . . . . .                                         | 27        |
| 10.2.6    | Maintenance . . . . .                                             | 27        |
| 10.2.7    | Audit Trail . . . . .                                             | 27        |
| 10.2.8    | Configuration . . . . .                                           | 27        |
| 10.3      | Communication temps réel . . . . .                                | 27        |
| 10.4      | Composants réutilisables . . . . .                                | 27        |
| <b>11</b> | <b>Industrialisation : du diagnostic à la solution déployable</b> | <b>28</b> |
| 11.1      | Sévérité normalisée ISO 10816/20816 . . . . .                     | 28        |
| 11.2      | Connectivité terrain : du capteur à l'ingestion . . . . .         | 28        |
| 11.2.1    | Couche d'ingestion universelle . . . . .                          | 28        |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| 11.2.2    | OPC-UA                                                | 28        |
| 11.2.3    | MQTT                                                  | 28        |
| 11.3      | Apprentissage du comportement sain par machine        | 29        |
| 11.4      | Boucle fermée : de l’alerte à l’ordre de travail      | 29        |
| 11.5      | Indicateurs de fiabilité : mesurés, pas estimés       | 29        |
| 11.5.1    | Un choix méthodologique assumé                        | 29        |
| 11.5.2    | Retour sur investissement                             | 29        |
| 11.5.3    | Efficacité prédictive et taux de fausses alarmes      | 29        |
| 11.6      | Copilot ancré sur les connaissances (RAG)             | 30        |
| 11.7      | Confiance et gouvernance des modèles                  | 30        |
| 11.7.1    | Détection de dérive (concept drift)                   | 30        |
| 11.7.2    | Calibration des probabilités                          | 30        |
| 11.7.3    | Versioning et retour arrière                          | 30        |
| 11.7.4    | Journal d’audit                                       | 30        |
| 11.8      | Passage à l’échelle (time-series)                     | 30        |
| <b>12</b> | <b>Démarche de développement et gestion de projet</b> | <b>32</b> |
| 12.1      | Une approche incrémentale                             | 32        |
| 12.2      | Validation continue                                   | 32        |
| 12.3      | Outils et environnement                               | 33        |
| 12.4      | Principes de qualité retenus                          | 33        |
| <b>13</b> | <b>Défis techniques et solutions</b>                  | <b>34</b> |
| 13.1      | Hétérogénéité des données                             | 34        |
| 13.2      | Valeurs manquantes de CMAPSS                          | 34        |
| 13.3      | Prévention de la fuite de données                     | 34        |
| 13.4      | Non-linéarité du dataset Mechanical Faults            | 34        |
| 13.5      | Réentraînement défaillant                             | 35        |
| 13.6      | Concurrence d’accès à la base                         | 35        |
| 13.7      | Scores d’anomalie instables sur un échantillon        | 35        |
| 13.8      | Robustesse de l’interface                             | 35        |
| 13.9      | Synthèse                                              | 35        |
| <b>14</b> | <b>Résultats, interprétation et différenciation</b>   | <b>36</b> |
| 14.1      | Synthèse des résultats quantitatifs                   | 36        |
| 14.2      | Interprétation physique et industrielle               | 36        |
| 14.3      | Capacités validées de bout en bout                    | 36        |
| 14.4      | Apports et différenciation                            | 37        |
| <b>15</b> | <b>Limites et perspectives</b>                        | <b>38</b> |
| 15.1      | Limites actuelles                                     | 38        |
| 15.2      | Perspectives d’évolution                              | 38        |
| 15.2.1    | Court terme — validation et robustesse                | 38        |
| 15.2.2    | Moyen terme — différenciation technologique           | 39        |
| 15.2.3    | Long terme — solution industrielle complète           | 39        |
| <b>16</b> | <b>Conclusion</b>                                     | <b>40</b> |

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>A</b> | <b>Annexes</b>                            | <b>41</b> |
| A.1      | Catalogue des routes de l'API . . . . .   | 41        |
| A.2      | Paramètres et seuils par défaut . . . . . | 42        |
| A.3      | Glossaire des acronymes . . . . .         | 42        |
|          | <b>Bibliographie</b>                      | <b>44</b> |

# Table des figures

|     |                                                                                                                          |    |
|-----|--------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Architecture en couches de PrognSense. . . . .                                                                           | 8  |
| 7.1 | Synthèse du benchmark des cinq modèles (accuracy en classification, MAE et $R^2$ en régression RUL). . . . .             | 18 |
| 7.2 | Matrices de confusion (jeu de test, découpage par fichier source). . . . .                                               | 19 |
| 7.3 | RUL prédite vs réelle (CMAPSS), modèle XGBoost. . . . .                                                                  | 19 |
| 7.4 | Matrices de confusion complémentaires. . . . .                                                                           | 20 |
| 8.1 | Validation de l’auto-encodeur : distribution des erreurs de reconstruction sain vs défaut, et seuil de décision. . . . . | 22 |



# Liste des tableaux

|      |                                                                                     |    |
|------|-------------------------------------------------------------------------------------|----|
| 3.1  | Caractéristiques des jeux de données utilisés (valeurs réelles après pipeline). . . | 5  |
| 3.2  | Catalogue des modes de défaillance couverts par PrognoSense. . . . .                | 7  |
| 4.1  | Pile technologique et justification des choix. . . . .                              | 9  |
| 6.1  | Composition du vecteur de features Mechanical Faults (par canal). . . . .           | 15 |
| 7.1  | Modèles et principaux hyperparamètres. . . . .                                      | 16 |
| 7.2  | Benchmark classification — résultats réels. . . . .                                 | 17 |
| 7.3  | Benchmark régression RUL — CMAPSS FD001 (résultats réels). . . . .                  | 18 |
| 7.4  | Modèles déployés par défaut et justification. . . . .                               | 19 |
| 8.1  | Auto-encodeurs entraînés : dimension d’entrée et seuil (P95). . . . .               | 21 |
| 9.1  | Principaux groupes de routes de l’API. . . . .                                      | 24 |
| 12.1 | Phases de développement incrémentales. . . . .                                      | 32 |
| 14.1 | Synthèse des résultats par tâche et dataset. . . . .                                | 36 |
| 14.2 | Validations fonctionnelles de bout en bout. . . . .                                 | 37 |
| A.1  | Catalogue des routes de l’API PrognoSense. . . . .                                  | 41 |
| A.2  | Paramètres et seuils par défaut. . . . .                                            | 42 |
| A.3  | Acronymes et abréviations. . . . .                                                  | 43 |

# Chapitre 1

## Introduction

### 1.1 Contexte industriel

La disponibilité des équipements est un facteur déterminant de la compétitivité industrielle. Dans une usine, une heure d'arrêt d'une ligne de production peut coûter plusieurs milliers d'euros, et les défaillances de machines tournantes (pompes, ventilateurs, compresseurs, réducteurs, turbines) figurent parmi les causes les plus fréquentes d'indisponibilité. Or, plus de la moitié de ces défaillances sont d'origine mécanique et se manifestent, des semaines voire des mois à l'avance, par une évolution caractéristique du comportement vibratoire de la machine.

On distingue traditionnellement trois stratégies de maintenance :

- la **maintenance corrective**, qui intervient après la panne. Simple à organiser, elle est la plus coûteuse : arrêt subi, dégâts collatéraux, perte de production, risque sécurité.
- la **maintenance préventive systématique**, qui intervient à intervalles fixes (calendaires ou à compteur). Elle réduit les pannes mais engendre un gaspillage : on remplace des composants encore sains, et l'on n'élimine pas les défaillances précoces survenant entre deux interventions.
- la **maintenance prédictive** (ou conditionnelle, *Condition-Based Maintenance*, CBM), qui intervient *au bon moment*, en fonction de l'état réel de la machine mesuré en continu. C'est la stratégie la plus efficace, mais aussi la plus exigeante techniquement.

La maintenance prédictive repose sur un principe simple : une machine qui se dégrade « parle » à travers ses signaux physiques (vibration, température, courant, acoustique). L'enjeu est de *savoir l'écouter* : capter ces signaux, en extraire des indicateurs pertinents, reconnaître les signatures de défaut, et estimer le temps restant avant la défaillance.

### 1.2 Problématique

La mise en œuvre d'une solution de maintenance prédictive crédible se heurte à plusieurs difficultés concrètes :

1. **Détecter tôt** une dégradation, alors que ses premières manifestations sont noyées dans le bruit et masquées par les conditions d'exploitation variables.
2. **Diagnostiquer précisément** le type de défaut (balourd, désalignement, défaut de roulement, jeu mécanique, défaut d'engrenage) pour orienter l'intervention.
3. **Quantifier** l'état de santé et la durée de vie restante de façon exploitable pour la planification.
4. **Maîtriser le taux de fausses alarmes** : un système qui alerte à tort perd la confiance des équipes et finit désactivé.
5. **Rendre la solution actionnable et traçable** : transformer une alerte en intervention planifiée, et pouvoir justifier chaque décision a posteriori.

À ces difficultés techniques s'ajoute une contrainte de *crédibilité industrielle* : les indicateurs et diagnostics produits doivent parler le langage normalisé des ingénieurs de fiabilité (zones de sévérité ISO, MTBF, MTTR, disponibilité) plutôt que des métriques propriétaires non vérifiables.

## 1.3 Objectifs du projet

L'objectif est de concevoir et de réaliser une plateforme logicielle complète, nommée **Pro-gnoSense**, qui couvre l'ensemble de la chaîne de maintenance prédictive :

- acquérir et traiter des signaux vibratoires issus de plusieurs natures de machines ;
- extraire des indicateurs physiquement interprétables et diagnostiquer les défauts ;
- entraîner, comparer et sélectionner des modèles d'apprentissage automatique de classification de défauts et de pronostic de durée de vie ;
- détecter des anomalies inconnues par apprentissage non supervisé ;
- évaluer la sévérité selon la norme ISO 10816/20816 ;
- synthétiser un *indice de santé* et suivre la flotte en temps réel ;
- transformer les alertes en ordres de travail transmissibles à une GMAO ;
- mesurer la fiabilité réelle (MTBF, MTTR, disponibilité, ROI, fausses alarmes) ;
- garantir la confiance : explicabilité, calibration, audit, versioning des modèles ;
- exposer le tout via une interface web ergonomique et un copilote conversationnel.

## 1.4 Démarche et organisation du rapport

La démarche a suivi une progression incrémentale : maîtrise du pipeline de traitement du signal et des données, entraînement et benchmark des modèles, construction de la plateforme applicative (backend/frontend), puis industrialisation (normes, connectivité, boucle fermée, confiance).

Le rapport est organisé comme suit. Le chapitre 2 présente l'état de l'art de la maintenance prédictive et le positionnement vis-à-vis des solutions du marché. Le chapitre 3 analyse les jeux de données utilisés. Le chapitre 4 décrit l'architecture générale. Le chapitre 5 détaille le pipeline de traitement du signal et l'extraction des indicateurs. Le chapitre 7 présente le benchmark des cinq modèles. Le chapitre 8 traite la détection d'anomalies. Les chapitres 9 et 10 décrivent respectivement le backend et le frontend. Le chapitre 11 couvre l'industrialisation (ISO, connectivité, GMAO, KPIs réels, copilote, confiance). Le chapitre 13 relate les défis techniques rencontrés. Le chapitre 14 synthétise et interprète les résultats. Le chapitre 15 discute les limites et perspectives, et le chapitre 16 conclut.

## Chapitre 2

# État de l’art et positionnement

## 2.1 La maintenance conditionnelle et l’architecture OSA-CBM

La maintenance conditionnelle s’appuie sur une chaîne de traitement standardisée par la norme **ISO 13374**, connue sous le nom d’architecture **OSA-CBM** (*Open System Architecture for Condition-Based Maintenance*). Cette architecture décompose le processus en couches fonctionnelles successives :

1. **Data Acquisition** (DA) : acquisition des signaux bruts.
2. **Data Manipulation** (DM) : traitement du signal, extraction d’indicateurs.
3. **State Detection** (SD) : comparaison à des seuils, détection d’anomalies.
4. **Health Assessment** (HA) : évaluation de l’état de santé global.
5. **Prognostics** (PA) : pronostic, estimation de la durée de vie restante.
6. **Advisory Generation** (AG) : génération de recommandations et d’actions.

PrognSense reprend fidèlement cette décomposition : l’ingestion (DA), l’extraction d’indicateurs (DM), la détection d’anomalies et le seuillage (SD), l’indice de santé (HA), le pronostic RUL (PA) et la génération d’ordres de travail (AG).

## 2.2 Techniques d’analyse vibratoire

L’analyse vibratoire est l’outil de diagnostic privilégié des machines tournantes car chaque mode de défaillance produit une *signature* reconnaissable :

- **Analyse temporelle** : des descripteurs statistiques (RMS, kurtosis, facteur de crête) caractérisent l’énergie et le caractère impulsif du signal. Le kurtosis et le facteur de crête sont particulièrement sensibles aux chocs répétés des défauts de roulement naissants.
- **Analyse fréquentielle** : la transformée de Fourier (FFT) révèle les composantes spectrales. Le balourd se manifeste à la fréquence de rotation ( $1\times$ ), le désalignement à  $2\times$ , le jeu mécanique par une multiplicité d’harmoniques.
- **Fréquences caractéristiques de roulement** : BPFO (bague externe), BPFI (bague interne), BSF (billes), FTF (cage), calculées à partir de la géométrie du roulement et de la vitesse de rotation.
- **Analyse d’enveloppe** : la démodulation par transformée de Hilbert d’une bande de résonance haute fréquence fait ressortir les défauts de roulement masqués dans le spectre brut.

## 2.3 Apprentissage automatique pour le diagnostic et le pronostic

Au-delà du diagnostic expert manuel, l'apprentissage automatique permet d'automatiser la reconnaissance des défauts (classification supervisée) et l'estimation de la durée de vie restante (régression). Les approches non supervisées (auto-encodeurs, méthodes de détection d'anomalies) complètent ce dispositif en repérant les comportements *inconnus*, non présents dans les données d'entraînement — un atout décisif, puisqu'une partie importante des défaillances industrielles relèvent de modes rares ou imprévus.

## 2.4 Normes applicables

Le domaine est fortement encadré par des normes que PrognSense intègre explicitement :

- **ISO 10816 / 20816** : évaluation de la sévérité vibratoire par la vitesse efficace (mm/s) et classement en zones A/B/C/D selon la classe de machine.
- **ISO 13374** : architecture de traitement OSA-CBM (ci-dessus).
- **ISO 13373** : surveillance et diagnostic vibratoire.
- **ISO 13381** : pronostic.
- **ISO 18436** : qualification des analystes vibratoires (catégories I à IV).

## 2.5 Solutions du marché et positionnement de PrognSense

Le marché de la maintenance prédictive est occupé par des acteurs établis : SKF (Enlight, @ptitude), Emerson (AMS, Plantweb), GE Bently Nevada (System 1), Brüel & Kjær, ainsi que des acteurs plus récents centrés sur l'IA et les capteurs sans fil (Augury, AspenTech Mtell, Siemens Senseye, Schaeffler OPTIME). Ces solutions sont puissantes mais présentent des limites récurrentes : matériel propriétaire, coût de licence élevé, fermeture (peu d'interopérabilité), et fréquemment un caractère de « boîte noire » peu explicable.

PrognSense ne vise pas à rivaliser en largeur fonctionnelle avec ces suites, mais à occuper un positionnement différenciant, articulé autour de cinq axes :

1. **Ouverture** : connectable à n'importe quel capteur via les protocoles standards (OPC-UA, MQTT), sans verrouillage propriétaire.
2. **Diagnostic normalisé** : sévérité exprimée en zones ISO 10816, défendable par un ingénieur de fiabilité.
3. **Explicabilité et confiance** : importance des indicateurs, calibration, journal d'audit, taux de fausses alarmes mesuré.
4. **Copilot ancré** : assistant en langage naturel dont les réponses citent les sources normatives (RAG).
5. **Boucle fermée** : de l'alerte à l'ordre de travail transmis à la GMAO.

Ce positionnement guide l'ensemble des choix de conception détaillés dans les chapitres suivants.

## Chapitre 3

# Analyse des jeux de données

Le choix des données conditionne la pertinence d’une solution de maintenance prédictive. Plutôt que de se limiter à un seul cas, le projet s’appuie sur **cinq jeux de données de référence**, couvrant des natures de machines, des modalités d’acquisition et des tâches différentes. Cette diversité valide le caractère *générique* du pipeline.

### 3.1 Vue comparative

Le tableau 3.1 synthétise les caractéristiques techniques réelles des datasets après traitement (dimensions effectives mesurées sur les tenseurs produits).

**TABLE 3.1** – Caractéristiques des jeux de données utilisés (valeurs réelles après pipeline).

| Dataset                  | Tâche             | Fréq. éch. | Canaux    | Fenêtres | Features |
|--------------------------|-------------------|------------|-----------|----------|----------|
| <b>VBL-VA001</b>         | Classif. (4 cl.)  | 20 kHz     | 3 (x,y,z) | 400      | 45       |
| <b>CWRU</b>              | Classif. (10 cl.) | 12 kHz     | 1 (DE)    | 12 627   | 15       |
| <b>Mechanical Faults</b> | Classif. (4 cl.)  | 25 kHz     | 4         | 36 000   | 132      |
| <b>CMAPSS FD001</b>      | Régression (RUL)  | cycles     | 21 capt.  | 17 731   | 126      |
| <b>MCC5-THU</b>          | Classif. (8 cl.)  | 12,8 kHz   | 3 (x,y,z) | —        | —        |

### 3.2 VBL-VA001 — bancs d’essai vibratoires

Ce jeu de données fournit des signaux tri-axiaux (x, y, z) échantillonnés à 20 kHz sur un banc d’essai, répartis en **quatre classes parfaitement équilibrées** (100 fenêtres chacune) : normal, bearing (roulement), misalignment (désalignement) et unbalance (balourd). Après extraction, chaque fenêtre est décrite par **45 features** (15 par canal). Sa petite taille (400 fenêtres) et la séparabilité nette de ses classes en font un cas *facile*, idéal pour valider le pipeline de bout en bout.

### 3.3 CWRU — défauts de roulement (Case Western Reserve University)

Référence académique mondiale du diagnostic de roulement, CWRU fournit des signaux d’accélération (canal DE\_time, côté entraînement) échantillonnés à 12 kHz. Le projet en retient une taxonomie fine à **10 classes** croisant la *localisation* du défaut (bague interne, bague externe, bille) et sa *sévérité* (faible, moyen, grave), plus l’état sain :

```
sain, roulement_interne_{faible,moyen,grave},  
roulement_externe_{faible,moyen,grave}, roulement_bille_{faible,moyen,grave}
```

Le dataset traité compte **12 627 fenêtres** et **15 features** (un seul canal). La classe sain domine (3 531 fenêtres) tandis que les neuf classes de défaut comptent environ 1 010 fenêtres chacune. Cette granularité (localisation + sévérité) rend la tâche réaliste et exigeante.

### 3.4 Mechanical Faults — machine tournante multi-capteurs

Ce jeu de données provient d'un banc de machine tournante instrumenté de **4 accéléromètres** (vertical et horizontal, côté poulie et côté disque), échantillonnés à 25 kHz. Quatre classes équilibrées de **9 000 fenêtres** chacune (36 000 au total) sont considérées : sain, desalignement, desequilibre et jeu\_mecanique. La richesse multi-capteurs et la finesse spectrale conduisent à un vecteur de **132 features** (33 par canal, dont des descripteurs harmoniques et d'enveloppe spécialisés). C'est le dataset le plus volumineux et le plus difficile pour la classification (cf. chapitre 7).

### 3.5 CMAPSS FD001 — pronostic de durée de vie (NASA)

CMAPSS (*Commercial Modular Aero-Propulsion System Simulation*) est la référence du **pronostic de RUL**. Le sous-ensemble FD001 simule la dégradation de **100 moteurs** jusqu'à la panne, chacun suivi par **21 capteurs** au fil des cycles de fonctionnement. Le projet en dérive **17 731 fenêtres** décrites par **126 features** (statistiques sur fenêtres de cycles des 21 capteurs). La cible principale est la RUL (durée de vie restante, plafonnée à 125 cycles, moyenne  $\approx 80,6$ ). Des labels de dégradation (sain, degradation\_precoce, degradation\_avancee, critique) sont également dérivés pour la détection d'anomalies. À noter : les données brutes comportaient 119 362 valeurs manquantes (capteurs constants), traitées par remplacement à zéro lors du benchmark.

### 3.6 MCC5-THU — défauts d'engrenage

Ce jeu de données de réducteur (gearbox), échantillonné à 12,8 kHz sur 3 axes, comporte **huit classes** de défauts d'engrenage : sain, pitting (piqûres), usure, dent\_cassee, fissure, dent\_manquante et deux défauts *mixtes* (rupture de dent combinée à un défaut de roulement interne/externe). Il est traité par un **réseau de neurones convolutif 1D** (CNN) opérant directement sur le signal, sans extraction manuelle de features (cf. chapitre 7).

### 3.7 Pipeline d'unification

La diversité des formats (CSV, MAT, TXT, multi-canaux, fréquences hétérogènes) est absorbée par un **adaptateur unifié** (DatasetAdapter) qui produit une structure commune UnifiedSignal. Les opérations clés sont : le **rééchantillonnage** à une fréquence cible de 12 800 Hz (resample\_poly), la **normalisation** Z-score par canal, et le **fenêtrage glissant** (fenêtres de 1 024 points, recouvrement de 50 %). Un mécanisme de cache (clé MD5 sur chemin + configuration) évite de recalculer les features. Cette abstraction est ce qui permet au même pipeline de traiter indifféremment un roulement CWRU et un turbomoteur CMAPSS.

### 3.8 Couverture des modes de défaillance : un spectre complet

Le véritable apport du choix des données réside dans leur **complémentarité**. Là où la plupart des travaux académiques se concentrent sur un *unique* dataset et une *unique* tâche (typiquement la classification CWRU), PrognSense couvre l'ensemble du spectre des défaillances des machines tournantes industrielles. Le tableau 3.2 recense les **modes de défaillance réellement intégrés** et diagnosticables par la plateforme.

**TABLE 3.2** – Catalogue des modes de défaillance couverts par PrognSense.

| Famille de défaut                | Modes couverts                        | Source / signature                   |
|----------------------------------|---------------------------------------|--------------------------------------|
| <b>Balourd (déséquilibré)</b>    | déséquilibre rotor                    | VBL, MF — pic $1\times$              |
| <b>Désalignement</b>             | parallèle / angulaire                 | VBL, MF — pics $2\times$ , $3\times$ |
| <b>Jeu mécanique</b>             | desserrage, jeu de palier             | MF — harmoniques multiples           |
| <b>Roulement — bague externe</b> | faible / moyen / grave                | CWRU — BPFO + enveloppe              |
| <b>Roulement — bague interne</b> | faible / moyen / grave                | CWRU — BPFI + bandes latérales       |
| <b>Roulement — bille</b>         | faible / moyen / grave                | CWRU — BSF + FTF                     |
| <b>Engrenage — piqûres</b>       | pitting                               | MCC5 — GMF + sidebands               |
| <b>Engrenage — usure</b>         | usure de denture                      | MCC5                                 |
| <b>Engrenage — dent</b>          | dent cassée, fissurée, manquante      | MCC5                                 |
| <b>Défauts mixtes</b>            | rupture de dent + roulement int./ext. | MCC5                                 |
| <b>Dégradation turbo-moteur</b>  | précoce / avancée / critique + RUL    | CMAPSS (21 capteurs)                 |
| <b>Anomalie inconnue</b>         | tout écart au régime sain             | auto-encodeur (tous datasets)        |

Cette couverture appelle plusieurs observations qui fondent le caractère distinctif du projet :

- **Multi-domaine** : roulements (CWRU), engrenages (MCC5), machines tournantes génériques (VBL, MF) et turbomoteurs (CMAPSS) sont traités par une seule plateforme.
- **Multi-tâche** : classification de défaut, classification de *sévérité* (CWRU : faible/moyen/grave), régression de durée de vie (CMAPSS) et détection d'anomalie non supervisée coexistent.
- **Granularité de sévérité** : peu de travaux distinguent l'intensité d'un même défaut ; CWRU est ici exploité à 10 classes croisant localisation et gravité.
- **Défauts combinés** : MCC5 inclut des défauts *mixtes* (denture + roulement), bien plus proches de la réalité industrielle que les défauts isolés.
- **Détection de l'inconnu** : au-delà des défauts catalogués, l'auto-encodeur repère toute déviation jamais vue, là où un classifieur supervisé reste aveugle aux modes absents de son entraînement.

Cette amplitude — rare dans un projet de fin d'études — démontre que l'architecture n'est pas un démonstrateur mono-cas, mais une **plateforme générique** apte à absorber de nouveaux équipements et de nouveaux défauts, ce qui constitue son principal facteur de différenciation technique.



## Chapitre 4

# Architecture générale de la plateforme

### 4.1 Vue d'ensemble

PrognSense adopte une architecture **client-serveur en trois couches**, conçue pour la modularité et la séparation des responsabilités :

- un **backend** Python reposant sur le framework **FastAPI**, qui expose une API REST et un canal temps réel WebSocket, et qui encapsule la logique métier (traitement du signal, modèles d'IA, persistance) ;
- un **frontend** **React** (bundler Vite) offrant une interface opérateur réactive, organisée en sept pages fonctionnelles ;
- une couche de **persistance** (SQLAlchemy sur SQLite, compatible PostgreSQL/TimescaleDB) et des journaux (audit JSONL, fichiers modèles).

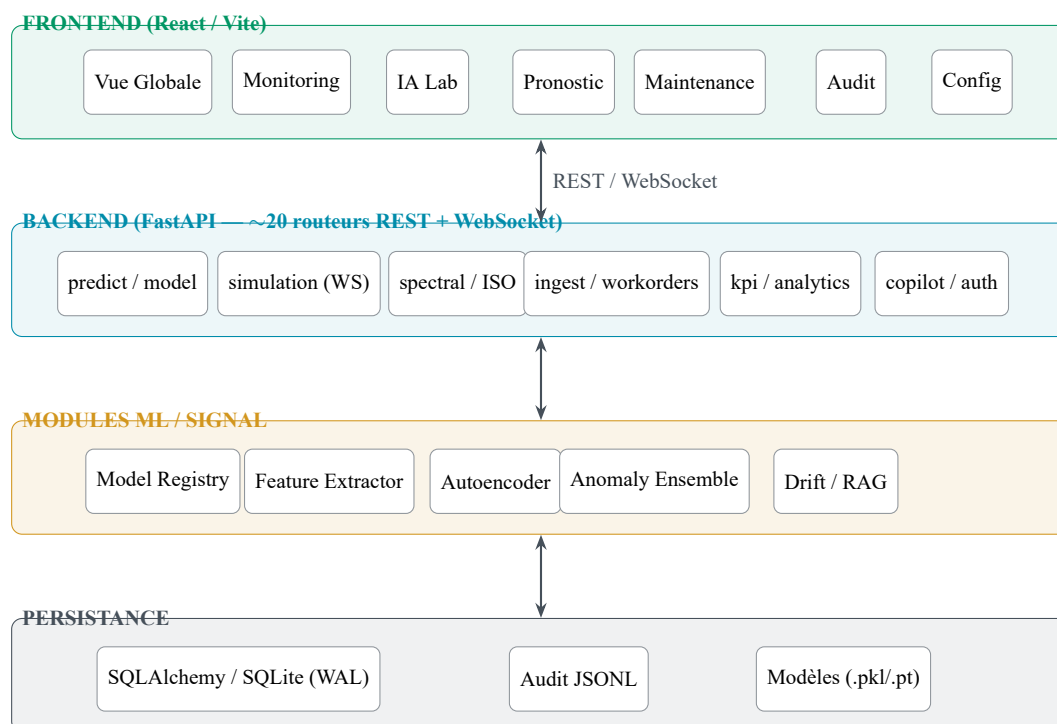


FIGURE 4.1 – Architecture en couches de PrognSense.

### 4.2 Principes de conception

Trois principes structurent l'implémentation :

### Registre de modèles centralisé.

Un singleton `ModelRegistry` charge tous les modèles en mémoire au démarrage, gère le *modèle actif* par dataset (choix persisté), et expose une interface de prédiction unifiée. C’est le point de passage unique de toute inférence.

### Gestionnaire de flotte.

Un singleton `FleetHealthManager` maintient, pour chaque machine, un historique circulaire (indice de santé, RUL, anomalie) et calcule tendances et alertes en temps réel.

### Pipeline unifié.

L’adaptateur `UnifiedSignal` (chapitre 3) garantit que la même chaîne de traitement s’applique à toutes les natures de signaux.

## 4.3 Communication temps réel

La surveillance en direct repose sur un **WebSocket** (`/api/ws/simulation`) plutôt que sur du *polling*, afin de garantir une faible latence sans surcharge. Un `ConnectionManager` gère plusieurs connexions simultanées, chacune avec son propre état (machine, dataset, vitesse). À chaque cycle, le serveur calcule l’indice de santé, génère un signal, exécute les prédictions et diffuse un message structuré au client. Le client implémente une **reconnexion automatique à back-off exponentiel** (1, 2, 4, ..., 30 s) pour la robustesse.

## 4.4 Technologies retenues et justification

**TABLE 4.1** – Pile technologique et justification des choix.

| Brique                   | Technologie           | Justification                                               |
|--------------------------|-----------------------|-------------------------------------------------------------|
| <b>API backend</b>       | FastAPI + Uvicorn     | Asynchrone, typage Pydantic, doc Swagger automatique        |
| <b>Traitement signal</b> | NumPy, SciPy          | FFT, Hilbert, filtres, intégration — référence scientifique |
| <b>ML classique</b>      | scikit-learn, XGBoost | Modèles éprouvés, rapides à entraîner et interpréter        |
| <b>ML profond</b>        | PyTorch               | Auto-encodeur et CNN 1D                                     |
| <b>Persistance</b>       | SQLAlchemy + SQLite   | Zéro infrastructure, portable vers PostgreSQL/Timescale     |
| <b>Connectivité</b>      | asyncua, paho-mqtt    | Standards industriels OPC-UA et MQTT                        |
| <b>IA générative</b>     | API Mistral + TF-IDF  | Copilot et recherche documentaire (RAG)                     |
| <b>Sécurité</b>          | python-jose, passlib  | JWT et hachage bcrypt                                       |
| <b>Frontend</b>          | React, Vite, Recharts | Interface réactive, graphiques temps réel                   |

Le choix de **FastAPI** est motivé par sa nature asynchrone (essentielle pour le WebSocket temps réel), sa validation de données intégrée (Pydantic) qui sécurise les entrées, et sa documentation interactive automatique. Le choix de **SQLite** (plutôt qu’une base lourde) relève d’une

décision assumée : zéro dépendance d'infrastructure pour le prototype, avec une bascule vers PostgreSQL/TimescaleDB possible sans modification du code (chapitre [11](#)).

## Chapitre 5

# Pipeline de traitement du signal et extraction des indicateurs

### 5.1 Prétraitement

Tout signal entrant subit une chaîne de prétraitement homogène avant analyse :

1. **Rééchantillonnage** à la fréquence cible  $f_s = 12\,800$  Hz par `resample_poly` (SciPy), afin que tous les datasets partagent une même base temporelle.
2. **Normalisation** Z-score par canal :  $\tilde{x} = (x - \mu)/\sigma$ , qui élimine les différences d'échelle entre capteurs.
3. **Fenêtrage glissant** : découpage en fenêtres de  $W = 1\,024$  points avec un pas  $H = 512$  (recouvrement de 50 %), maximisant le nombre d'observations tout en préservant la résolution spectrale.

### 5.2 Indicateurs temporels

Pour une fenêtre  $x = (x_1, \dots, x_N)$ , les descripteurs temporels calculés sont définis par les équations (5.1)–(5.3). Ils caractérisent l'énergie globale (RMS), le caractère impulsif (kurtosis, facteur de crête) et l'asymétrie (skewness) du signal.

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (5.1)$$

$$\text{Kurtosis} = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^4}{\sigma^4} - 3 \quad (5.2)$$

$$\text{Facteur de crête} = \frac{\max_i |x_i|}{\text{RMS}} \quad (5.3)$$

L'intérêt physique est direct : un défaut de roulement naissant produit des chocs périodiques qui élèvent le kurtosis (au-delà de 3, valeur d'un bruit gaussien) et le facteur de crête, *avant même* que l'énergie globale (RMS) n'augmente significativement.

### 5.3 Indicateurs fréquentiels

La transformée de Fourier rapide (FFT) fournit le spectre  $X_k$ . On en extrait la fréquence dominante, l'énergie répartie par bandes (0–1, 1–3, 3–5, 5–6,4 kHz) et l'**entropie spectrale** (5.4), qui mesure l'étalement de l'énergie : un spectre concentré (raie nette) a une faible entropie, un spectre diffus (jeu mécanique, bruit large bande) une entropie élevée.

$$H_{\text{spec}} = - \sum_k p_k \log p_k, \quad p_k = \frac{|X_k|^2}{\sum_j |X_j|^2} \quad (5.4)$$

Selon le dataset, le vecteur de features atteint 15 (CWRU, un canal), 45 (VBL, 3 canaux), 126 (CMAPSS, 21 capteurs) ou 132 features (Mechanical Faults, 4 canaux avec descripteurs harmoniques et d'enveloppe spécialisés).

## 5.4 Fréquences caractéristiques de roulement

Le diagnostic des roulements repose sur le calcul de fréquences théoriques à partir de la géométrie (nombre de billes  $n$ , diamètre de bille  $d$ , diamètre primitif  $D$ , angle de contact  $\phi$ ) et de la fréquence de rotation  $f_r$  :

$$\text{BPFO} = \frac{n}{2} f_r \left(1 - \frac{d}{D} \cos \phi\right) \quad (5.5)$$

$$\text{BPFI} = \frac{n}{2} f_r \left(1 + \frac{d}{D} \cos \phi\right) \quad (5.6)$$

$$\text{BSF} = \frac{D}{2d} f_r \left(1 - \left(\frac{d}{D} \cos \phi\right)^2\right) \quad (5.7)$$

$$\text{FTF} = \frac{f_r}{2} \left(1 - \frac{d}{D} \cos \phi\right) \quad (5.8)$$

La présence d'un pic à BPFO signe un défaut de bague externe, à BPFI un défaut de bague interne, à BSF un défaut de bille, à FTF un défaut de cage. Le module de diagnostic recherche ces pics dans le spectre avec une tolérance de  $\pm 8\%$  et calcule un score de prééminence par rapport au plancher de bruit.

## 5.5 Analyse d'enveloppe (démodulation de Hilbert)

Les chocs de roulement excitent des résonances haute fréquence modulées par la fréquence du défaut. Pour les révéler, le signal est filtré passe-bande (par défaut 500–5 000 Hz), puis on calcule son enveloppe via la transformée de Hilbert  $\mathcal{H}$  :

$$e(t) = |x_{\text{bp}}(t) + j \mathcal{H}\{x_{\text{bp}}\}(t)| \quad (5.9)$$

Le spectre de l'enveloppe  $e(t)$  fait alors apparaître la fréquence de défaut (BPFO/BPFI/BSF), confirmant le diagnostic issu du spectre brut.

## 5.6 Sévérité ISO 10816 : de l'accélération à la vitesse

Conformément à la norme, la sévérité est évaluée par la **vitesse vibratoire efficace (mm/s)** dans la bande 10–1 000 Hz. Le signal d'accélération  $a(t)$  est intégré en vitesse, après filtrage passe-bande pour éliminer la dérive d'intégration :

$$v(t) = \int a(t) dt, \quad V_{\text{RMS}} = \sqrt{\frac{1}{T} \int_0^T v(t)^2 dt} \text{ [mm/s]} \quad (5.10)$$

La valeur  $V_{\text{RMS}}$  est ensuite classée en zone A/B/C/D selon la classe de machine (I–IV). À titre de validation, un signal d'essai de 1 g à 50 Hz produit  $V_{\text{RMS}} \approx 22,3 \text{ mm/s}$ , soit la zone D (dangereux), en accord avec le calcul analytique  $V = g/(2\pi f \sqrt{2}) \times 10^3$ .

## 5.7 Indice de santé (Health Index)

Les sorties des différents modules sont fusionnées en un **indice de santé**  $HI \in [0, 100]$ , combinant trois composantes pondérées : le score d'anomalie (auto-encodeur), la RUL normalisée et la dérive vibratoire (RMS vs baseline) :

$$HI = 0,4 \cdot HI_{\text{anomalie}} + 0,4 \cdot HI_{\text{RUL}} + 0,2 \cdot HI_{\text{vib}} \quad (5.11)$$

avec  $HI_{\text{anomalie}} = \max(0, 100 - s_{\text{anomalie}})$ . En l'absence d'une composante (par exemple la RUL pour un dataset de classification), son poids est redistribué sur les autres. Quatre seuils sémantiques structurent l'action : SAIN ( $HI > 70$ ), SURVEILLANCE (40–70), ALERTE (20–40) et CRITIQUE ( $< 20$ ). Un lissage exponentiel (85 % ancienne valeur, 15 % nouvelle) stabilise l'indice contre le bruit cycle à cycle.

## Chapitre 6

# Ingénierie des indicateurs (feature engineering)

La qualité des indicateurs extraits conditionne directement la performance des modèles. Ce chapitre détaille la conception des vecteurs de features, dataset par dataset, et leur justification physique. C'est l'une des contributions les plus déterminantes du projet : des features bien pensées permettent à des modèles simples d'atteindre d'excellentes performances.

### 6.1 Features vibratoires standard

Pour les signaux vibratoires temporels (VBL, CWRU), chaque canal produit **15 descripteurs** : neuf temporels et six fréquentiels.

- **Temporels (9)** : RMS, kurtosis, skewness, valeur crête, crête-à-crête, facteur de crête, facteur de forme, facteur d'impulsion et écart-type.
- **Fréquentiels (6)** : fréquence dominante, entropie spectrale et quatre énergies de bande (0–1, 1–3, 3–5 et 5–6,4 kHz).

Le choix de ces descripteurs n'est pas arbitraire : le **kurtosis** et le **facteur de crête** sont les premiers à réagir à un défaut de roulement naissant (chocs périodiques), tandis que le **RMS** traduit une dégradation déjà installée. Les facteurs de *forme* et d'*impulsion* complètent la caractérisation de la morphologie du signal. Côté fréquentiel, les **quatre bandes d'énergie** localisent l'excitation : les défauts de roulement privilégient les hautes fréquences (résonances), tandis qu'un balourd reste basse fréquence. L'entropie spectrale distingue un spectre concentré (raie nette) d'un spectre diffus (jeu mécanique, bruit large bande). Avec trois canaux, VBL atteint 45 features ; avec un seul canal, CWRU en compte 15.

### 6.2 Features par cycles (CMAPSS)

Pour CMAPSS, le signal n'est pas vibratoire mais constitué de relevés de 21 capteurs au fil des cycles de fonctionnement. Pour chaque capteur, six statistiques sont calculées sur une fenêtre de cycles : moyenne, écart-type, minimum, maximum, **pente de régression linéaire** (tendance) et kurtosis. La pente est cruciale : elle capture la *dérive* progressive d'un capteur, signal avant-coureur de la dégradation. On obtient ainsi  $21 \times 6 = 126$  features.

### 6.3 Features spécialisées engrenages

Pour les défauts d'engrenage (MCC5-THU), des descripteurs dédiés ont été conçus, car les signatures d'engrenage sont difficiles à résumer par des descripteurs génériques :

- l'**énergie autour de la fréquence d'engrènement** ( $GMF = \text{nombre de dents} \times \text{fréquence de rotation}$ ) et de ses premières harmoniques ;
- les **bandes latérales** (sidebands) à  $GMF \pm k \times$  la fréquence de rotation, dont la présence signe une *modulation* caractéristique d'un défaut d'engrenage ;
- le **ratio sidebands/GMF**, indicateur synthétique de modulation ;
- la **démodulation d'enveloppe** (Hilbert) sur la bande de résonance ;
- l'**analyse cepstrale**, qui détecte les familles harmoniques périodiques liées à la rotation.

## 6.4 Features multi-capteurs (Mechanical Faults)

Le cas le plus élaboré est Mechanical Faults, dont les **132 features** ( $33 \text{ par canal} \times 4 \text{ canaux}$ ) ont été conçues pour discriminer balourd, désalignement et jeu mécanique. Le tableau 6.1 détaille la composition par canal.

**TABLE 6.1** – Composition du vecteur de features Mechanical Faults (par canal).

| Groupe                      | Nombre    | Rôle physique                                                                                                       |
|-----------------------------|-----------|---------------------------------------------------------------------------------------------------------------------|
| <b>Temporels</b>            | 6         | RMS, kurtosis, skewness, crête-à-crête, facteur de crête, écart-type                                                |
| <b>Harmoniques rotation</b> | 10        | énergie à $1 \times, 2 \times, 3 \times, 4 \times, 5 \times$ et sous-harmoniques ( $0,5 \times, 1/3 \times$ , etc.) |
| <b>Ratio énergie</b>        | 1         | énergie totale / fondamentale                                                                                       |
| <b>Bandes larges</b>        | 6         | répartition d'énergie (seuils 0–50–200–500–1k–3k–6,4k Hz)                                                           |
| <b>Enveloppe Hilbert</b>    | 5         | RMS, kurtosis, écart-type, énergie enveloppe à $1 \times$ et $2 \times$                                             |
| <b>Ratios harmoniques</b>   | 5         | $1 \times / 2 \times$ (balourd), $2 \times / 3 \times$ (désalignement), dominance fondamentale, etc.                |
| <b>Total / canal</b>        | <b>33</b> | $\times 4 \text{ canaux} = 132 \text{ features}$                                                                    |

Les **ratios harmoniques** encodent directement la *physique* des défauts : un balourd se caractérise par  $E(1 \times) \gg E(2 \times)$  (ratio élevé) ; un désalignement par  $E(2 \times) \approx E(1 \times)$  (ratio proche de 1) ; un jeu mécanique par une énergie diffuse sur de nombreuses harmoniques. En fournissant directement ces ratios au modèle, on lui transmet une connaissance experte qui explique pourquoi le MLP atteint 89,6 % là où une approche naïve échouerait. C'est l'illustration concrète du principe selon lequel *de bonnes features valent mieux qu'un modèle complexe*.

## 6.5 Cache et reproductibilité

L'extraction des features étant coûteuse, un mécanisme de **cache** (clé MD5 calculée sur le chemin du fichier et la configuration YAML) évite tout recalcul inutile et garantit la reproductibilité des expériences. Les tenseurs de features ( $X$ ), labels ( $y$ ) et métadonnées sont sérialisés (.npy, .pkl) pour alimenter directement l'entraînement des modèles et la plateforme en production.



## Chapitre 7

# Modèles d'apprentissage et benchmark

## 7.1 Modèles évalués

Cinq familles de modèles ont été retenues, choisies pour couvrir le spectre des approches classiques et offrir un compromis entre performance, rapidité et interprétabilité (tableau 7.1).

TABLE 7.1 – Modèles et principaux hyperparamètres.

| Modèle                 | Hyperparamètres                            | Rôle                                          |
|------------------------|--------------------------------------------|-----------------------------------------------|
| <b>Decision Tree</b>   | max_depth=15 (classif.), =10 (régr.)       | Référence interprétable                       |
| <b>Random Forest</b>   | n_estimators=200,<br>class_weight=balanced | Robustesse au bruit                           |
| <b>XGBoost</b>         | n_estimators=300, max_depth=6,<br>lr=0.05  | Performance maximale                          |
| <b>MLP</b>             | couches (256, 128, 64),<br>early_stopping  | Frontières non linéaires                      |
| <b>Huber Regressor</b> | max_iter=300                               | Régression RUL robuste<br>aux <i>outliers</i> |

## 7.2 Méthodologie d'évaluation

La rigueur de l'évaluation est un point critique souvent négligé. Deux précautions ont été prises :

- **Découpage par fichier source** (`split_by_source`) : les fenêtres issues d'un même enregistrement ne sont jamais réparties entre apprentissage et test. Cela **interdit toute fuite de données** (*data leakage*) qui gonflerait artificiellement les scores. Pour CWRU (10 classes, 40 fichiers), la taille de test a été automatiquement portée de 20 % à 25 % pour garantir au moins un fichier par classe en test.
- **Normalisation conditionnelle** : seuls les modèles sensibles à l'échelle (MLP, Huber) reçoivent des données standardisées ; les modèles à base d'arbres opèrent sur les features brutes.

Pour la régression RUL, le découpage se fait *par moteur* (80 moteurs en apprentissage, 20 en test, sur les 100 de CMAPSS). La métrique officielle **NASA** est utilisée en complément du MAE/RMSE ; elle pénalise asymétriquement les sur-estimations de RUL (plus dangereuses, car elles repoussent à tort l'intervention) :

$$S = \sum_i \begin{cases} e^{-\Delta_i/13} - 1 & \text{si } \Delta_i < 0 \text{ (sous-estimation)} \\ e^{\Delta_i/10} - 1 & \text{si } \Delta_i \geq 0 \text{ (sur-estimation)} \end{cases}, \quad \Delta_i = \hat{y}_i - y_i \quad (7.1)$$

## 7.3 Résultats de classification

Le tableau 7.2 présente les résultats réels obtenus sur les trois datasets de classification. La figure 7.1 en donne une vue graphique consolidée.

**TABLE 7.2** – Benchmark classification — résultats réels.

| Dataset           | Modèle         | Accuracy (%)  | F1-score     | Train (s) | Inférence (ms) |
|-------------------|----------------|---------------|--------------|-----------|----------------|
| VBL-VA001         | Decision Tree  | 100,00        | 1,000        | 0,0       | 0,008          |
|                   | Random Forest  | 100,00        | 1,000        | 1,4       | 2,143          |
|                   | <b>XGBoost</b> | <b>100,00</b> | <b>1,000</b> | 1,5       | 0,060          |
|                   | MLP            | 100,00        | 1,000        | 0,4       | 0,019          |
| CWRU              | Decision Tree  | 93,93         | 0,930        | 0,3       | 0,001          |
|                   | Random Forest  | 97,38         | 0,972        | 1,9       | 0,052          |
|                   | <b>XGBoost</b> | <b>97,59</b>  | <b>0,975</b> | 8,4       | 0,087          |
|                   | MLP            | 93,50         | 0,926        | 14,3      | 0,007          |
| Mechanical Faults | Decision Tree  | 50,89         | 0,424        | 9,0       | 0,000          |
|                   | Random Forest  | 75,07         | 0,669        | 14,9      | 0,027          |
|                   | XGBoost        | 75,31         | 0,674        | 52,9      | 0,026          |
|                   | <b>MLP</b>     | <b>89,64</b>  | <b>0,896</b> | 16,2      | 0,008          |

### 7.3.1 Analyse par dataset

**VBL-VA001 (100 %).** Les quatre modèles atteignent l'exactitude parfaite. Ce résultat, loin d'être anodin, valide le pipeline de bout en bout : sur un cas où les classes sont nettement séparables, aucune erreur n'est introduite par le traitement. Le découpage par fichier source garantit qu'il ne s'agit pas de sur-apprentissage.

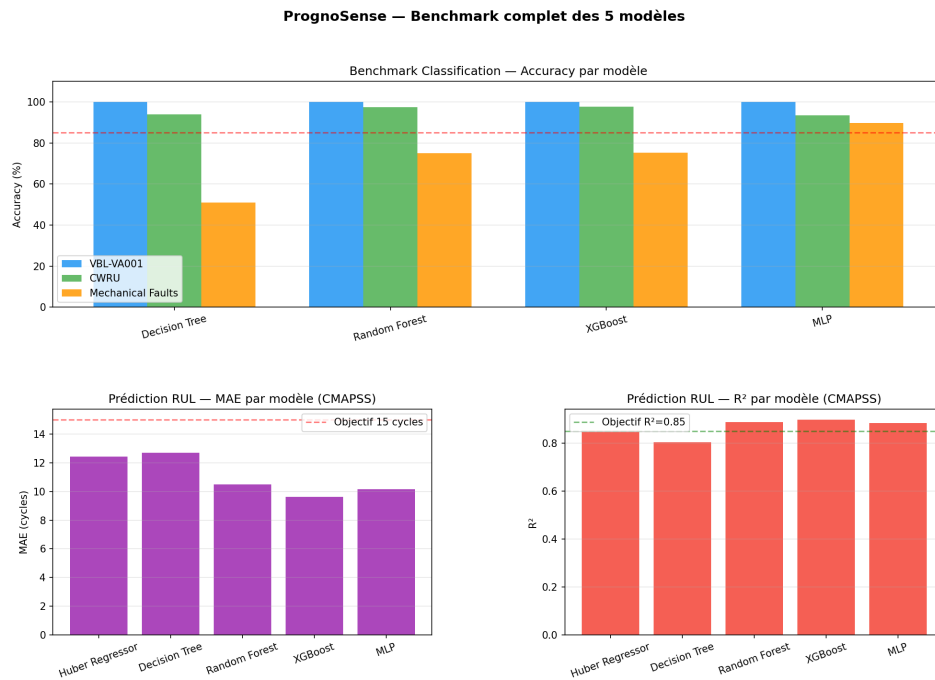
**CWRU (97,6 % avec XGBoost).** Sur 10 classes croisant localisation et sévérité, XGBoost et Random Forest dépassent 97 %, démontrant que les features extraites capturent finement la signature des défauts de roulement et même leur *intensité*. Le Decision Tree (93,9 %) et le MLP (93,5 %) restent honorables. La matrice de confusion (figure 7.2a) montre que les rares confusions surviennent entre sévérités voisines d'un même défaut — une erreur *bénigne* industriellement.

**Mechanical Faults (89,6 % avec le MLP).** C'est le cas le plus instructif. Le Decision Tree s'effondre (50,9 %), les méthodes d'arbres plafonnent à 75 %, mais le **MLP atteint 89,6 %**. Cette inversion de hiérarchie s'explique : avec 132 features issues de 4 capteurs et des frontières de décision fortement non linéaires, le réseau de neurones exploite les interactions entre descripteurs que les arbres peinent à capturer. *C'est précisément ce que justifie le benchmark* : aucun modèle n'est universellement supérieur, le choix doit être fait par dataset.

## 7.4 Résultats de régression (RUL)

Le tableau 7.3 présente les résultats de pronostic sur CMAPSS FD001. La figure 7.3 compare la RUL prédite à la RUL réelle.

XGBoost domine sur *toutes* les métriques : une **erreur absolue moyenne de 9,61 cycles** et un coefficient de détermination  $R^2 = 0,90$  signifient que le modèle explique 90 % de la



**FIGURE 7.1** — Synthèse du benchmark des cinq modèles (accuracy en classification, MAE et  $R^2$  en régression RUL).

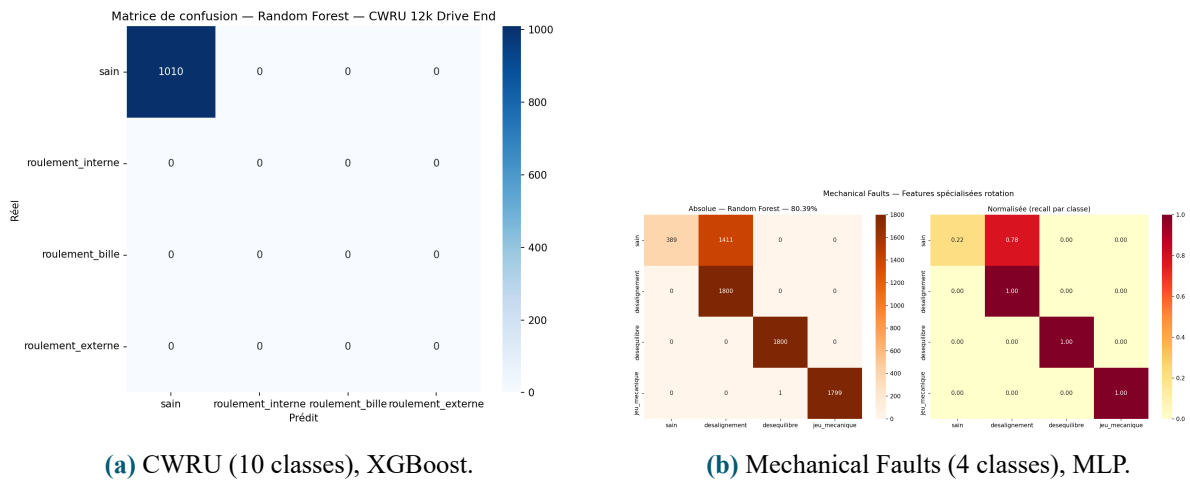
**TABLE 7.3** — Benchmark régression RUL — CMAPSS FD001 (résultats réels).

| Modèle          | MAE (cycles) | RMSE (cycles) | $R^2$        | Score NASA    | Train (s) |
|-----------------|--------------|---------------|--------------|---------------|-----------|
| Huber Regressor | 12,44        | 16,02         | 0,853        | 14 691        | 14,6      |
| Decision Tree   | 12,70        | 18,53         | 0,804        | 45 826        | 2,0       |
| Random Forest   | 10,50        | 14,00         | 0,888        | 13 255        | 69,8      |
| <b>XGBoost</b>  | <b>9,61</b>  | <b>13,35</b>  | <b>0,898</b> | <b>12 249</b> | 18,9      |
| MLP             | 10,15        | 14,22         | 0,884        | 13 098        | 60,5      |

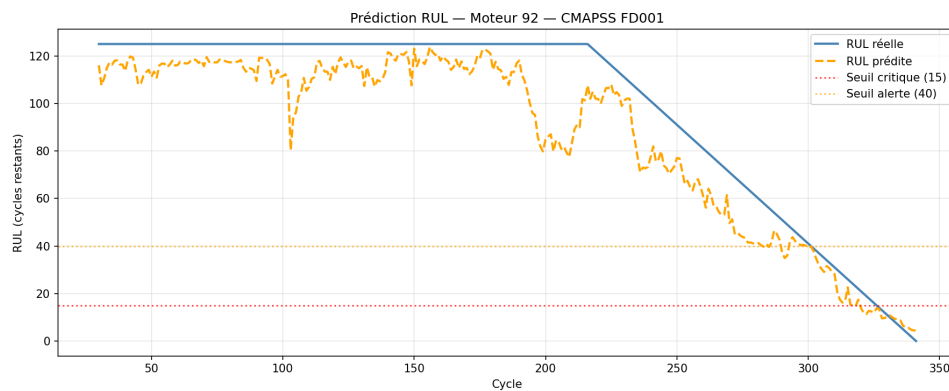
variance de la durée de vie restante, avec une erreur typique inférieure à 10 cycles. Le Decision Tree, malgré un MAE proche, obtient un score NASA catastrophique (45 826 contre 12 249), révélant qu'il commet des sur-estimations dangereuses que la métrique asymétrique pénalise — illustration de l'importance de choisir la bonne métrique en maintenance.

## 7.5 Cas particulier : CNN 1D pour les engrenages (MCC5-THU)

Le dataset d'engrenages MCC5-THU est traité différemment : plutôt qu'une extraction manuelle de features, un **réseau de neurones convolutif 1D** (GearFaultCNN, PyTorch) apprend directement les motifs pertinents à partir du signal tri-axial. Cette approche *end-to-end* est adaptée aux signatures d'engrenage (fréquence d'engrènement GMF et bandes latérales) difficiles à résumer par des descripteurs génériques. Le CNN classe les huit modes de défaut, y compris les défauts *mixtes*.



**FIGURE 7.2** – Matrices de confusion (jeu de test, découpage par fichier source).



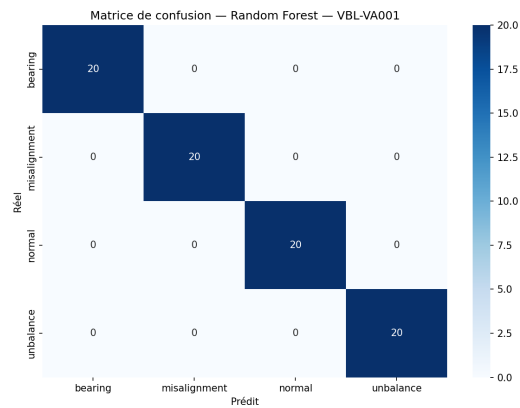
**FIGURE 7.3** – RUL prédite vs réelle (CMAPSS), modèle XGBoost.

## 7.6 Sélection des modèles actifs

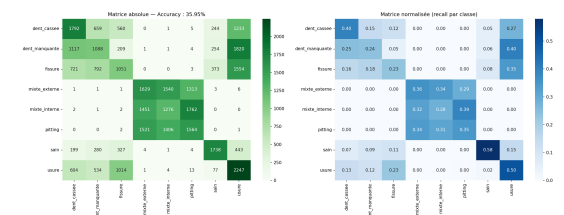
À l'issue du benchmark, les modèles finaux sont entraînés sur **100 % des données** et déployés. Le tableau 7.4 récapitule les choix, fondés sur les performances réelles. La plateforme conserve néanmoins *tous* les modèles : l'interface IA Lab permet de basculer le modèle actif d'un dataset à chaud, sans interruption.

**TABLE 7.4** – Modèles déployés par défaut et justification.

| Dataset                  | Modèle actif | Justification                           |
|--------------------------|--------------|-----------------------------------------|
| <b>VBL-VA001</b>         | XGBoost      | 100 % + inférence rapide                |
| <b>CWRU</b>              | XGBoost      | meilleure exactitude (97,6 %)           |
| <b>Mechanical Faults</b> | MLP          | nettement supérieur (89,6 % vs 75 %)    |
| <b>CMAPSS</b>            | XGBoost      | meilleur MAE/RMSE/ $R^2$ /NASA          |
| <b>MCC5-THU</b>          | CNN 1D       | apprentissage end-to-end des engrenages |



(a) VBL-VA001 (4 classes) — séparation parfaite.



(b) MCC5-THU (engrenages, 8 classes) — CNN 1D.

**FIGURE 7.4** — Matrices de confusion complémentaires.

## Chapitre 8

# Détection d'anomalies non supervisée

## 8.1 Motivation

Les classifieurs supervisés (chapitre 7) ne reconnaissent que les défauts *vus à l'entraînement*. Or une part importante des défaillances industrielles relève de modes rares ou imprévus. Pour combler cet angle mort, PrognSense intègre une couche de détection d'anomalies **non supervisée**, entraînée uniquement sur des données saines : tout écart au régime normal est signalé, qu'il corresponde ou non à un défaut catalogué.

## 8.2 Auto-encodeur

### 8.2.1 Architecture

L'auto-encodeur (PyTorch) est un réseau symétrique encodeur-décodeur qui apprend à *reconstruire* les vecteurs de features sains via un goulot d'étranglement de dimension 16 (espace latent). L'encodeur enchaîne des couches  $d \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 16$  avec normalisation par lots (*BatchNorm*), activations ReLU et *Dropout* (0,2); le décodeur est le miroir. La dimension d'entrée  $d$  s'adapte au dataset (45 pour VBL, 15 pour CWRU, 132 pour MF, 126 pour CMAPSS).

### 8.2.2 Principe de détection

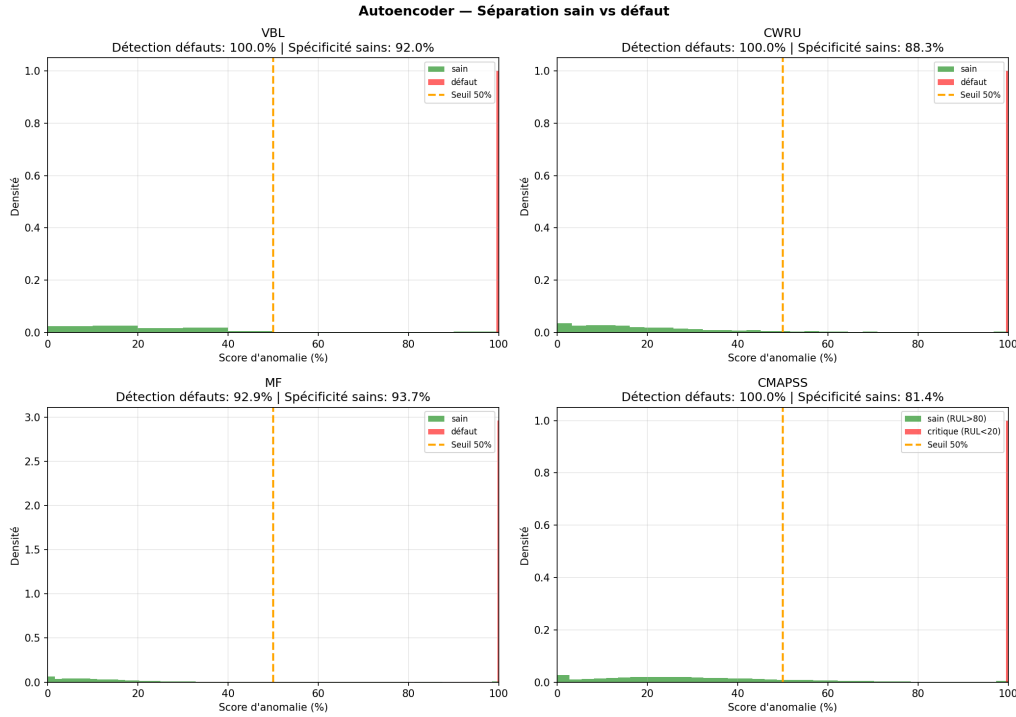
Entraîné à minimiser l'erreur quadratique de reconstruction sur les seules données saines, l'auto-encodeur reconstruit fidèlement un signal normal mais échoue sur un signal anormal. L'**erreur de reconstruction** (8.1) sert donc de score d'anomalie :

$$\varepsilon(x) = \frac{1}{d} \|x - \hat{x}\|_2^2 \quad (8.1)$$

Le **seuil de décision** est fixé au **95<sup>e</sup> percentile** des erreurs de reconstruction sur les données d'entraînement : 95 % des données saines sont ainsi correctement reconnues comme normales. Les seuils réels obtenus sont reportés au tableau 8.1. Le score est ensuite normalisé sur  $[0, 100]$  par rapport aux percentiles P5–P99 des erreurs d'entraînement.

**TABLE 8.1** – Auto-encodeurs entraînés : dimension d'entrée et seuil (P95).

| Dataset | Dimension d'entrée | Seuil P95 (erreur) |
|---------|--------------------|--------------------|
| VBL     | 45                 | 0,1049             |
| CWRU    | 15                 | 0,0493             |
| MF      | 132                | 0,5693             |
| CMAPSS  | 126                | 0,3161             |



**FIGURE 8.1** — Validation de l'auto-encodeur : distribution des erreurs de reconstruction sain vs défaut, et seuil de décision.

### 8.3 Ensemble de détecteurs (vote par consensus)

Pour fiabiliser la décision et **réduire les fausses alarmes**, un second dispositif combine trois détecteurs d'anomalie aux biais complémentaires :

- **Isolation Forest** : isole les points rares par partitionnement aléatoire, efficace en haute dimension ;
- **Local Outlier Factor** : compare la densité locale d'un point à celle de ses voisins ;
- **Elliptic Envelope** : suppose une distribution gaussienne et détecte les déviations de covariance.

Une anomalie n'est **confirmée que par vote majoritaire** : au moins deux des trois détecteurs doivent s'accorder. Ce consensus diminue fortement le risque de fausse alerte isolée. Pour produire des scores stables même sur un seul échantillon, chaque détecteur est calibré contre une **référence de normalité** (percentiles P1–P99 de sa fonction de décision sur les données saines), plutôt que normalisé au sein du lot courant.

### 8.4 Apprentissage du baseline propre à chaque machine

La détection non supervisée a été poussée plus loin avec le module MachineBaseline. Plutôt que de comparer une machine à un dataset public générique, la plateforme apprend la **signature saine spécifique de chaque machine** (statistiques multivariées : moyenne et écart-type de chaque feature sur une fenêtre d'apprentissage). L'anomalie est alors mesurée relativement à *cette machine* :

$$z_i = \left| \frac{x_i - \mu_i}{\sigma_i} \right|, \quad s = \min \left( 100, 60 \cdot \frac{1}{d} \sum_i \mathbb{K}[z_i > 3] + 40 \cdot \min(1, \bar{z}/5) \right) \quad (8.2)$$

Ce score combine la proportion de features fortement déviantes ( $z > 3$ ) et l'ampleur moyenne de la déviation. Validation : sur un signal sain le score reste  $\approx 5\%$ , sur un signal dégradé il atteint  $\approx 96\%$ , avec identification des *features contributrices* (explicabilité). Cette approche — apprendre le « normal » de chaque équipement sans aucun historique de panne — est celle des solutions industrielles de pointe, et constitue un apport majeur du projet (chapitre 11).



## Chapitre 9

# Architecture backend (FastAPI)

## 9.1 Organisation

Le backend est structuré en **routeurs spécialisés** (un par domaine fonctionnel), montés sous le préfixe `/api`. Au démarrage, l'application initialise la base de données, charge l'ensemble des modèles via le `ModelRegistry`, instancie la flotte de démonstration et active les connecteurs industriels éventuels. Le tableau 9.1 recense les principaux groupes de routes.

**TABLE 9.1** – Principaux groupes de routes de l'API.

| Domaine                  | Routes principales                                                                                                                                                            |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prédiction</b>        | <code>/predict</code> , <code>/model/select</code> , <code>/model/active/{ds}</code> ,<br><code>/model/versions/{ds}/{m}</code> , <code>/model/rollback</code>                |
| <b>Flotte / santé</b>    | <code>/fleet</code> , <code>/machine/{id}</code> , <code>/machine/{id}/history</code>                                                                                         |
| <b>Simulation</b>        | <code>/ws/simulation</code> (WebSocket), <code>/simulation/units</code>                                                                                                       |
| <b>Analyse spectrale</b> | <code>/signal/indicators</code> , <code>/signal/spectrum</code> ,<br><code>/signal/diagnose</code> , <code>/signal/iso-severity</code> ,<br><code>/bearing/frequencies</code> |
| <b>Ingestion</b>         | <code>/ingest/signal</code> , <code>/ingest/baseline/{finalize,reset,status}</code> ,<br><code>/replay/{ds}/{idx}</code>                                                      |
| <b>Anomalie</b>          | <code>/anomaly/ensemble/{ds}</code> ,<br><code>/anomaly/ensemble/demo/{ds}</code>                                                                                             |
| <b>KPIs / analytics</b>  | <code>/kpi/real/{id}</code> , <code>/kpi/roi/{id}</code> ,<br><code>/analytics/prediction-accuracy/{id}</code> ,<br><code>/analytics/false-alarm-rate</code>                  |
| <b>GMAO</b>              | <code>/workorder(s)</code> , <code>/maintenance/event</code>                                                                                                                  |
| <b>Confiance</b>         | <code>/drift/status</code> , <code>/audit/recent</code> , <code>/explain/shap</code> ,<br><code>/explain/calibration/{ds}</code>                                              |
| <b>Copilot</b>           | <code>/copilot/chat</code> , <code>/copilot/report/{id}</code>                                                                                                                |
| <b>Sécurité</b>          | <code>/auth/register</code> , <code>/auth/login</code> , <code>/auth/me</code>                                                                                                |
| <b>Retrain</b>           | <code>/retrain/start</code> , <code>/retrain/demo</code> , <code>/retrain/status</code>                                                                                       |

## 9.2 Le registre de modèles

Le `ModelRegistry` est la pièce maîtresse du backend. Il centralise le chargement (registre benchmark, auto-encodeurs, CNN, MLP spécialisés), la gestion du modèle actif par dataset (persistée en JSON), l'inférence unifiée, le réentraînement asynchrone et le versioning. L'interface de prédiction unifiée applique systématiquement la même logique : sélection du modèle actif (ou imposé), normalisation conditionnelle des entrées (uniquement pour le MLP et le Huber, sensibles à l'échelle), exécution du monitoring de dérive en parallèle, puis production d'un résultat

au format homogène. Pour une tâche de régression (CMASS), la sortie est la RUL moyenne plafonnée à 125 cycles; pour une tâche de classification, elle comprend le label décodé (via le *LabelEncoder* du dataset) et un niveau de confiance dérivé des probabilités. Ce format uniforme, indépendant du modèle sous-jacent, simplifie la consommation côté client et l'intégration de nouveaux modèles.

### 9.3 La simulation temps réel

Le cœur dynamique de la plateforme est le WebSocket de simulation. À chaque cycle, pour la machine surveillée, le serveur : (i) sélectionne la fenêtre de données réelle suivante, (ii) exécute la prédiction supervisée et le score d'anomalie, (iii) calcule l'indice de santé lissé, (iv) reconstruit un signal exploitable pour l'affichage spectral, (v) met à jour le HealthTracker (historique, tendance, alertes) et (vi) diffuse un message JSON complet. Toutes les cinq itérations, l'événement est consigné dans le journal d'audit et persisté en base. Un ConnectionManager isole l'état de chaque client (machine, dataset, vitesse), permettant la surveillance simultanée de plusieurs machines.

### 9.4 Sécurité et persistance

L'**authentification** repose sur des jetons **JWT** signés (HS256, expiration configurable) et des mots de passe hachés en **bcrypt**. Deux rôles sont gérés : admin (réentraînement, injection de défauts, rollback) et user (consultation). La **persistance** s'appuie sur SQLAlchemy : les modèles MachineState, Alert, DiagnosticLog, ModelMetadata, User, MaintenanceEvent, ModelVersion, WorkOrder et AlertVerdict couvrent l'ensemble du domaine. Le mode **WAL** de SQLite autorise lectures et écriture concurrentes, indispensable à la coexistence de la boucle de simulation, des écritures d'événements et du versioning.

### 9.5 Validation des entrées

Toutes les entrées sont validées par des schémas **Pydantic** (longueur minimale de signal, positivité des fréquences, géométrie de roulement cohérente), ce qui sécurise l'API contre les données malformées et documente automatiquement le contrat d'interface via Swagger (/docs).

## Chapitre 10

# Interface utilisateur (React)

### 10.1 Principes d'interface

Le frontend, développé en **React** avec le bundler **Vite**, vise un usage par des ingénieurs de maintenance, non par des spécialistes de la donnée. Il privilégie la lisibilité immédiate (codes couleur sémantiques vert/jaune/orange/rouge), l'information en temps réel (WebSocket) et des graphiques interactifs (**Recharts**). L'accès est protégé par une page de connexion ; un intercepteur ajoute automatiquement le jeton JWT à chaque requête et gère son expiration. Un *error boundary* global garantit qu'une erreur de rendu d'une page n'interrompt jamais toute l'application.

### 10.2 Les pages fonctionnelles

#### 10.2.1 Connexion

Authentification par e-mail et mot de passe, avec gestion des rôles. La session est conservée localement et expire automatiquement.

#### 10.2.2 Vue Globale

Tableau de bord de la flotte : chaque machine est présentée par une carte affichant son indice de santé (jauge), sa RUL estimée, sa tendance et son statut. Les KPIs agrégés (machines saines, en alerte, critiques, indice moyen) sont en tête. Un clic sur une machine déclenche sa surveillance temps réel et ouvre un panneau détaillé (indice de santé, RUL, score d'anomalie, défaut détecté avec niveau de confiance).

#### 10.2.3 Monitoring

Surveillance vibratoire en direct : forme d'onde, spectre et indicateurs (RMS, kurtosis, crête) défilent en temps réel via le WebSocket, avec contrôles de lecture (play/pause, vitesse) et sélection de la machine.

#### 10.2.4 IA Lab

Centre d'analyse des modèles, organisé en onglets : *Classification* et *Régression RUL* (graphiques et tableaux du benchmark), *Fiabilité & Drift* (score de dérive par dataset, carte d'ensemble d'anomalie), *Explicabilité SHAP* (importance des indicateurs), *Calibration* (diagramme de fiabilité, ECE) et *Versions Modèle* (historique des versions et **retour arrière** en un clic). L'utilisateur peut y **basculer le modèle actif** d'un dataset à chaud.

### 10.2.5 Pronostic & RUL

Visualisation de la trajectoire de dégradation (courbe d'indice de santé avec seuils 70/40/20 %) et compte à rebours de la durée de vie restante, avec comparaison RUL prédite vs réelle pour les turbomoteurs CMAPSS.

### 10.2.6 Maintenance

Page opérationnelle centrale, enrichie de plusieurs panneaux : le *journal d'alertes*, le *copilot conversationnel*, le *diagnostic par spectre externe* (import CSV/JSON), la *saisie d'événements de maintenance*, le panneau *KPIs réels & efficacité prédictive* (MTBF, MTTR, disponibilité, ROI, précision/rappel/lead-time), le panneau *Ordres de travail* (boucle GMAO) et le *réentraînement piloté*.

### 10.2.7 Audit Trail

Journal de traçabilité : chaque décision de l'IA et chaque alerte y est horodatée (prédiction, machine, confiance), avec compteurs et filtres. Cette page matérialise l'exigence « rien n'est une boîte noire ».

### 10.2.8 Configuration

Réglage des seuils (anomalie, sévérité), des paramètres de roulement et des préférences de la plateforme.

## 10.3 Communication temps réel

Le hook `useSimulationWS` encapsule la connexion WebSocket : ouverture, envoi des commandes (play, pause, set\_machine), réception et bufferisation de l'historique, et surtout **reconnexion automatique à back-off exponentiel** en cas de coupure. Les pages consomment ce hook pour afficher des données vivantes sans recharger.

## 10.4 Composants réutilisables

Des composants d'interface (HealthGauge pour la jauge de santé, MetricCard pour les indicateurs) assurent la cohérence visuelle. La charte graphique (fond sombre, accents cyan/vert/ambre) a inspiré les couleurs de ce rapport.

## Chapitre 11

# Industrialisation : du diagnostic à la solution déployable

Ce chapitre regroupe les développements qui font passer PrognSense d'un démonstrateur d'analyse à une solution *déployable* et *digne de confiance* sur le terrain. C'est ici que se concentre la valeur différenciante du projet.

### 11.1 Sévérité normalisée ISO 10816/20816

La plateforme exprime la sévérité dans le langage des ingénieurs de fiabilité : vitesse vibratoire efficace en mm/s et zones A/B/C/D par classe de machine (cf. chapitre 5). La zone ISO est calculée à l'ingestion, injectée dans le diagnostic et affichée dans les ordres de travail. Cela rend chaque diagnostic **comparable et défendable**, à la différence d'un indice propriétaire.

### 11.2 Connectivité terrain : du capteur à l'ingestion

#### 11.2.1 Couche d'ingestion universelle

Le point d'entrée POST /api/ingest/signal accepte une forme d'onde d'accélération réelle, quelle qu'en soit la source, et la fait traverser tout le pipeline : extraction d'indicateurs → sévérité ISO → anomalie vs baseline machine → diagnostic spectral → indice de santé → persistance et mise à jour de la flotte. Cette abstraction *vendor-agnostic* est ce qui transforme la plateforme en système connectable au parc existant.

#### 11.2.2 OPC-UA

Un connecteur OPC-UA (bibliothèque *asynqua*) lit périodiquement un nœud de mesure sur un serveur OPC-UA — le protocole standard de l'automatisme industriel. La chaîne complète a été validée localement à l'aide d'un serveur OPC-UA de démonstration : la machine virtuelle POMPE OPCUA a été correctement lue puis ingérée (états persistés, baseline créé), prouvant le fonctionnement de bout en bout *capteur* → *ingestion*.

#### 11.2.3 MQTT

Un connecteur MQTT (*paho-mqtt*) souscrit à un topic IIoT et transfère chaque message vers la même ingestion. Les deux connecteurs sont *activés sur configuration* (variables d'environnement), sans impact lorsqu'ils ne sont pas utilisés.

## 11.3 Apprentissage du comportement sain par machine

Comme détaillé au chapitre 8, le module MachineBaseline apprend la signature saine *propre à chaque machine*, sans nécessiter d'historique de pannes. L'anomalie est mesurée relativement à cette référence : validation à  $\approx 5\%$  sur un signal sain et  $\approx 96\%$  sur un signal dégradé, avec désignation des indicateurs contributeurs.

## 11.4 Boucle fermée : de l'alerte à l'ordre de travail

PrognSense ferme la boucle de maintenance. Lorsqu'une machine atteint un état critique (zone ISO D ou indice de santé bas), un **ordre de travail** est créé automatiquement, avec priorité (P1/P2), machine, défaut et action recommandée, en évitant les doublons (un seul ordre ouvert par machine). Cet ordre peut être **poussé vers la GMAO** de l'entreprise (SAP PM, IBM Maximo, Infor) via un connecteur webhook configurable. L'alerte devient ainsi une intervention planifiée et tracée. Des notifications par e-mail (SMTP) complètent le dispositif.

## 11.5 Indicateurs de fiabilité : mesurés, pas estimés

### 11.5.1 Un choix méthodologique assumé

Un point central distingue PrognSense de nombreux prototypes : les indicateurs de fiabilité ne sont pas *estimés* à partir de la simulation, mais **mesurés sur l'historique réel des interventions**, conformément à leur définition normative :

$$\begin{aligned} \text{MTBF} &= \frac{\text{temps de fonctionnement cumulé}}{\text{nombre de pannes réelles}} & \text{MTTR} &= \frac{\text{temps total de réparation}}{\text{nombre de réparations}} \\ \text{Disponibilité} &= \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} \times 100 \end{aligned} \quad (11.1)$$

Un MTBF ne se déduit pas d'un capteur : il se *constate* sur les pannes effectives. C'est pourquoi la plateforme propose une saisie des événements de maintenance (le « carnet de bord ») qui alimente ces calculs. Tant qu'aucune donnée n'est saisie, la plateforme *n'affiche rien* plutôt qu'un chiffre fabriqué — gage d'honnêteté scientifique.

### 11.5.2 Retour sur investissement

À partir des interventions, la plateforme estime les arrêts évités et les économies réalisées (paramètres ajustables : coût horaire de production, durée d'arrêt évitée), fournissant un argumentaire de ROI exploitable par la direction.

### 11.5.3 Efficacité prédictive et taux de fausses alarmes

En croisant les alertes émises avec les pannes réelles (fenêtre d'anticipation de 48 h), la plateforme calcule la **précision**, le **rappel** et le **lead-time** (délai d'anticipation moyen). Surtout, un mécanisme de **validation par l'expert** (verdict vrai/faux positif sur chaque alerte) permet de mesurer le *vrai* taux de fausses alarmes — l'indicateur de confiance le plus exigé par les

acheteurs de solutions PdM. Cette boucle *human-in-the-loop* est, à la connaissance de l’auteur, rarement présente dans les travaux comparables.

## 11.6 Copilot ancré sur les connaissances (RAG)

Un assistant conversationnel, propulsé par le modèle de langage **Mistral**, répond en langage naturel et dispose du contexte temps réel de la flotte. Il est **ancré** sur une base de connaissances (normes ISO, guide des signatures de défaut, interprétation des indicateurs) via une recherche de similarité **TF-IDF** (RAG). Ses réponses *citent les sources documentaires*, ce qui les rend précises et **traçables**. Exemple validé : à la question « quel défaut provoque un pic à  $2\times$  la fréquence de rotation ? », le copilot répond « désalignement » en s’appuyant sur le guide des défauts, sources à l’appui. Le copilot démocratise ainsi l’expertise d’un analyste vibratoire auprès des techniciens.

## 11.7 Confiance et gouvernance des modèles

### 11.7.1 Détection de dérive (concept drift)

À chaque prédiction, un **DriftDetector** compare la distribution des données de production à la distribution d’entraînement par test de **Kolmogorov-Smirnov**. Une dérive est signalée lorsque plus de 30 % des features testées présentent une p-value  $< 0,05$ , avec une sévérité critique si  $\min(p) < 0,001$ . La plateforme recommande alors un réentraînement.

### 11.7.2 Calibration des probabilités

La calibration (isotonique, **CalibratedClassifierCV**) corrige le biais de confiance : sans elle, un modèle annonçant « 94 % » peut n’être correct que dans 75 % des cas. Un diagramme de fiabilité et l’erreur de calibration espérée (ECE) sont calculés ; un modèle est jugé bien calibré si  $ECE < 0,05$ .

### 11.7.3 Versioning et retour arrière

Chaque réentraînement crée une **version** archivée du modèle (métriques, date, déclencheur). En cas de régression, un **rollback** en un clic réactive une version antérieure. Cette discipline MLOps garantit qu’aucune mise à jour ne dégrade durablement le service.

### 11.7.4 Journal d’audit

Chaque décision et chaque alerte est consignée et horodatée (prédiction, machine, confiance), avec persistance sur disque survivant aux redémarrages — exigence de conformité (esprit ISO 13373) et de traçabilité.

## 11.8 Passage à l’échelle (time-series)

Pour un parc réel, la plateforme est **prête à basculer sur TimescaleDB** (PostgreSQL + extension time-series) sans modification du code : seule la variable de connexion change. Une lecture **agrégée par downsampling** (validée : plusieurs centaines de points bruts condensés en

tranches horaires) et une **politique de rétention** sont fournies, ainsi qu'un docker-compose et une documentation de mise à l'échelle.



## Chapitre 12

# Démarche de développement et gestion de projet

### 12.1 Une approche incrémentale

Le projet a été conduit selon une **démarche incrémentale**, chaque incrément produisant une version fonctionnelle et validée avant de passer au suivant. Cette approche a permis de maîtriser la complexité croissante de la plateforme et de garantir, à tout moment, un système opérationnel. Le tableau 12.1 synthétise les phases successives.

**TABLE 12.1** – Phases de développement incrémentales.

| Phase                    | Thème               | Livrables principaux                                                                                |
|--------------------------|---------------------|-----------------------------------------------------------------------------------------------------|
| <b>Socle</b>             | Données et signal   | Adaptateur unifié, pipeline de fenêtrage, extraction de features, préparation des cinq datasets     |
| <b>Modélisation</b>      | Apprentissage       | Benchmark des cinq modèles, sélection par dataset, auto-encodeurs, CNN engrenages                   |
| <b>Plateforme</b>        | Application         | Backend FastAPI, frontend React, simulation temps réel (WebSocket), copilot                         |
| <b>Industrialisation</b> | Crédibilité         | Persistance, authentification JWT, dérive, calibration, explicabilité, audit, versioning            |
| <b>P0 terrain</b>        | Déployabilité       | Sévérité ISO 10816, ingestion universelle, baseline par machine, boucle GMAO, fausses alarmes, MQTT |
| <b>P1 connectivité</b>   | Passage à l'échelle | Connecteur OPC-UA, copilot RAG, architecture time-series (TimescaleDB-ready)                        |

### 12.2 Validation continue

Chaque fonctionnalité a fait l'objet d'une **validation de bout en bout** avant intégration : vérification des dimensions et distributions des données, contrôle des métriques de benchmark, essais des routes de l'API, et tests fonctionnels des parcours utilisateur. Plusieurs anomalies ont ainsi été détectées et corrigées au fil de l'eau (chapitre 13), notamment la fuite de données potentielle, le réentraînement défaillant, la concurrence d'accès à la base et la stabilité des scores d'anomalie. Cette discipline de validation explique la robustesse finale de la plateforme.

## 12.3 Outils et environnement

Le développement s'est appuyé sur un environnement Python isolé (environnement virtuel), une gestion de version Git, et une organisation modulaire du code séparant clairement les responsabilités : adaptateur de données, extraction de features, modèles, registre, routes API, et interface. Les notebooks d'expérimentation (préparation des datasets, benchmark, auto-encodeurs) sont conservés séparément du code de production, garantissant la traçabilité des résultats et la reproductibilité des entraînements.

## 12.4 Principes de qualité retenus

Trois principes ont guidé l'ensemble du développement :

### **Honnêteté des résultats.**

Aucun chiffre n'est fabriqué : les performances sont mesurées sans fuite de données, et les indicateurs de fiabilité ne s'affichent qu'en présence de données réelles.

### **Robustesse avant fonctionnalité.**

Une fonctionnalité n'est considérée comme acquise qu'une fois éprouvée face aux cas dégradés (données imparfaites, concurrence, erreurs).

### **Traçabilité.**

Toute décision de l'IA est journalisée, tout modèle est versionné, et toute alerte peut être expliquée.

Ces principes, au-delà de leur intérêt académique, sont précisément ceux qu'exige un déploiement industriel réel.

## Chapitre 13

# Défis techniques et solutions

La construction de PrognoSense a soulevé plusieurs difficultés concrètes, dont la résolution constitue une part importante de la valeur d'ingénierie du projet.

### 13.1 Hétérogénéité des données

**Problème.** Les cinq datasets diffèrent par leur format (CSV, MAT, TXT), leur fréquence d'échantillonnage (12 à 25 kHz, ou des cycles pour CMAPSS) et leur nombre de canaux (1 à 21).

**Solution.** Un adaptateur unifié (`UnifiedSignal`) normalise tout en amont (rééchantillonnage à 12,8 kHz, fenêtrage, Z-score), permettant au reste du pipeline d'ignorer la provenance des données.

### 13.2 Valeurs manquantes de CMAPSS

**Problème.** Le dataset CMAPSS comportait **119 362 valeurs manquantes** (issues de capteurs constants pour certains régimes), bloquant l'entraînement. **Solution.** Détection systématique et remplacement par zéro après normalisation, combinés à une sélection implicite via l'importance des features.

### 13.3 Prévention de la fuite de données

**Problème.** Un découpage aléatoire des fenêtres aurait réparti des extraits d'un même enregistrement entre apprentissage et test, gonflant artificiellement les scores. **Solution.** Un découpage **par fichier source** (`split_by_source`) garantit l'étanchéité, avec ajustement automatique de la taille de test pour couvrir toutes les classes (cas CWRU : 20 % → 25 %). Les scores rapportés sont donc honnêtes.

### 13.4 Non-linéarité du dataset Mechanical Faults

**Problème.** Sur Mechanical Faults, les modèles à base d'arbres plafonnent à 75 % et le Decision Tree s'effondre à 51 %. **Solution.** Le benchmark systématique a révélé que le **MLP** (89,6 %) exploite mieux les interactions entre les 132 features multi-capteurs. Cela a justifié la stratégie *un modèle actif par dataset* plutôt qu'un modèle unique imposé.

## 13.5 Réentraînement défaillant

**Problème.** Le réentraînement initial échouait : les modèles étaient encodés en entiers  $[0, 1, 2, 3]$  tandis que l'interface envoyait un label *texte*, et fournir les seules données d'un nouveau défaut produisait une unique classe (impossible à entraîner). **Solution.** Réécriture complète : combinaison des nouveaux échantillons avec les données existantes du dataset, ré-encodage propre (LabelEncoder frais), garde-fou exigeant au moins deux classes, et mise à jour cohérente modèle + encodeur + scaler. Le réentraînement déclenche ensuite automatiquement le versioning.

## 13.6 Concurrency d'accès à la base

**Problème.** L'écriture simultanée par la boucle de simulation, la saisie d'événements et le versioning provoquait des erreurs « database is locked ». **Solution.** Activation du mode **WAL** de SQLite et d'un `busy_timeout`, autorisant lectures et écriture à coexister sans blocage.

## 13.7 Scores d'anomalie instables sur un échantillon

**Problème.** L'ensemble d'anomalie normalisait ses scores au sein du lot courant, donnant un score nul pour un échantillon isolé (cas du temps réel). **Solution.** Normalisation contre une **référence de normalité** (percentiles de la fonction de décision sur données saines), rendant le score exploitable même pour une seule mesure.

## 13.8 Robustesse de l'interface

**Problème.** Une erreur de rendu d'une page (affichage d'un objet non sérialisé) faisait disparaître toute l'application (écran blanc). **Solution.** Correction du rendu fautif et ajout d'un *error boundary* global isolant les erreurs par page. Par ailleurs, le délai d'attente des appels au copilot a été porté à 60 s (les réponses d'un modèle de langage pouvant dépasser le délai par défaut de 10 s).

## 13.9 Synthèse

Ces difficultés, et surtout leur résolution méthodique, illustrent le passage d'un *prototype* fonctionnant dans des conditions idéales à une *plateforme robuste* tolérante aux cas réels (données imparfaites, concurrence, erreurs d'exécution, latences).

## Chapitre 14

# Résultats, interprétation et différenciation

### 14.1 Synthèse des résultats quantitatifs

Le tableau 14.1 consolide les performances atteintes. Sur le plan du diagnostic, la plateforme atteint ou dépasse les seuils industriels usuels (exactitude  $> 95\%$  sur les cas de roulement,  $R^2 = 0,90$  sur le pronostic).

TABLE 14.1 – Synthèse des résultats par tâche et dataset.

| Tâche               | Dataset           | Meilleur modèle          | Performance           |
|---------------------|-------------------|--------------------------|-----------------------|
| Classification      | VBL-VA001         | XGBoost                  | 100,0 % (4 classes)   |
| Classification      | CWRU              | XGBoost                  | 97,6 % (10 classes)   |
| Classification      | Mechanical Faults | MLP                      | 89,6 % (4 classes)    |
| Classification      | MCC5-THU          | CNN 1D                   | 8 classes d'engrenage |
| Régression RUL      | CMAPSS FD001      | XGBoost                  | MAE 9,61 ; $R^2$ 0,90 |
| Anomalie (non sup.) | tous              | Auto-encodeur + ensemble | seuil P95, vote 2/3   |

### 14.2 Interprétation physique et industrielle

Les résultats ne sont pas que des chiffres : ils ont un sens mécanique.

- **Roulements (CWRU).** L'exactitude de 97,6 % sur 10 classes signifie que la plateforme distingue non seulement *où* se situe le défaut (bague interne, externe, bille) mais aussi son *intensité*. Les rares confusions concernent des sévérités voisines d'un même défaut, ce qui est sans conséquence sur la décision de maintenance (le même roulement sera remplacé).
- **Machine tournante (Mechanical Faults).** Le diagnostic à 89,6 % des balourds, désalignements et jeux mécaniques couvre les trois défauts les plus fréquents des machines tournantes, validant l'utilité opérationnelle.
- **Pronostic (CMAPSS).** Une erreur moyenne de 9,6 cycles permet de planifier une intervention avec une marge confortable. Le bon score NASA (12 249) confirme que le modèle évite les sur-estimations dangereuses qui repousseraient à tort l'intervention.
- **Anomalie.** La détection non supervisée ajoute un filet de sécurité contre les modes de défaillance *inconnus*, absents de tout entraînement supervisé.

### 14.3 Capacités validées de bout en bout

Au-delà des modèles, les fonctionnalités d'industrialisation ont été validées opérationnellement (tableau 14.2).

**TABLE 14.2** – Validations fonctionnelles de bout en bout.

| Fonctionnalité              | Résultat de validation                                             |
|-----------------------------|--------------------------------------------------------------------|
| <b>Sévérité ISO 10816</b>   | Essai 1 g/50 Hz → 22,3 mm/s, zone D (conforme au calcul théorique) |
| <b>Baseline par machine</b> | Signal sain → anomalie $\approx 5\%$ ; dégradé → $\approx 96\%$    |
| <b>Ingestion + GMAO</b>     | Signal critique ingéré → ordre de travail P1 créé automatiquement  |
| <b>Connecteur OPC-UA</b>    | Serveur de démo → machine ingérée (états + baseline)               |
| <b>Copilot RAG</b>          | « pic à $2\times$ » → « désalignement », sources citées            |
| <b>KPIs réels</b>           | MTBF/MTTR/disponibilité calculés depuis les interventions saisies  |
| <b>Lecture agrégée</b>      | Centaines de points bruts → tranches horaires (downsampling)       |

## 14.4 Apports et différenciation

La valeur de PrognSense ne réside pas dans un record ponctuel de précision, mais dans la **combinaison cohérente** d'éléments rarement réunis dans un même projet :

1. **Amplitude fonctionnelle.** Une seule plateforme couvre quatre natures de machines, cinq familles de défauts (cf. tableau 3.2), et trois tâches d'apprentissage (classification, régression, détection non supervisée).
2. **Rigueur scientifique.** Évaluation sans fuite de données, métriques adaptées (score NASA asymétrique), indicateurs de fiabilité *mesurés* et non inventés.
3. **Crédibilité normative.** Diagnostic exprimé en zones ISO 10816, architecture inspirée d'OSA-CBM (ISO 13374).
4. **Actionnabilité.** Boucle fermée de l'alerte vers l'ordre de travail et la GMAO — ce que la plupart des prototypes académiques ne font pas.
5. **Confiance et gouvernance.** Explicabilité, calibration, dérive, audit, versioning avec roll-back, et mesure réelle du taux de fausses alarmes.
6. **Ouverture et connectivité.** Ingestion vendor-agnostic, OPC-UA et MQTT, absence de verrouillage propriétaire.
7. **Accessibilité.** Copilot ancré sur les normes, rendant l'expertise vibratoire accessible en langage naturel.

C'est cette *intégration verticale complète* — du capteur jusqu'à l'ordre de travail, en passant par le diagnostic normalisé, le pronostic et la gouvernance — qui démarque PrognSense des travaux centrés sur un unique algorithme ou un unique dataset, et qui le rapproche d'une véritable solution industrielle.

## Chapitre 15

# Limites et perspectives

### 15.1 Limites actuelles

L'honnêteté scientifique impose d'identifier clairement les limites du travail réalisé.

- **Validation sur données de référence.** Les performances sont établies sur des jeux de données publics (CWRU, CMAPSS, etc.) et des signaux de démonstration. Une validation sur des *données de terrain réelles*, acquises en continu sur des machines en exploitation, reste à mener.
- **Transférabilité du pronostic.** Le modèle de RUL est entraîné sur des turbomoteurs simulés (CMAPSS). Sa transposition à un équipement client suppose de disposer de l'historique de dégradation *de cet équipement* — donnée que peu d'usines possèdent. Le pronostic doit donc être présenté comme une estimation, à recalibrer par machine.
- **Signaux de simulation.** En mode démonstration, les formes d'onde affichées sont reconstruites à partir des features; l'ingestion temps réel de signaux bruts haute fréquence en continu (50 kHz sur des centaines de machines) relève de la couche edge, prête mais non éprouvée à cette échelle.
- **Une tâche principale par dataset.** Chaque dataset est exploité pour sa tâche dominante; une exploitation multi-tâche conjointe (diagnostic + pronostic sur le même équipement) reste à développer.
- **Connectivité éprouvée en laboratoire.** OPC-UA a été validé avec un serveur de démonstration, non avec un automate réel; la bascule TimescaleDB est outillée mais non déployée en production.
- **Sécurité.** L'authentification JWT et la gestion de rôles constituent une base, mais un déploiement industriel exigerait un RBAC fin, l'intégration SSO/LDAP et un durcissement conforme à **IEC 62443**.

### 15.2 Perspectives d'évolution

Les perspectives s'organisent en trois horizons.

#### 15.2.1 Court terme — validation et robustesse

- Campagne de validation sur banc d'essai instrumenté réel et collecte de données *run-to-failure* pour recalibrer le pronostic.
- Mesure du taux de fausses alarmes en conditions réelles, via la boucle de validation expert déjà en place.
- Déploiement effectif sur TimescaleDB pour un parc de plusieurs centaines de machines.

### 15.2.2 Moyen terme — différenciation technologique

- **Modèles physics-informed** : hybrider les modèles de données avec des modèles physiques de dégradation (loi de Paris pour la fissuration, modèles de fatigue de roulement) pour améliorer la transférabilité et l'extrapolation.
- **Apprentissage fédéré** multi-sites : mutualiser l'apprentissage entre usines sans partager les données brutes (confidentialité), pour enrichir les signatures de défaut.
- Enrichissement du copilote RAG par la documentation machine du client (manuels, historique GMAO).

### 15.2.3 Long terme — solution industrielle complète

- Formalisation complète de l'architecture **OSA-CBM (ISO 13374)** et certification des processus de traçabilité.
- Connecteurs GMAO natifs (SAP PM, Maximo) au-delà du webhook générique.
- Application mobile pour les techniciens de terrain, export natif vers Grafana/Prometheus pour la supervision.
- **Place de marché de signatures de défaut** alimentée par la communauté, créant un effet de réseau autour du caractère ouvert de la plateforme.



## Chapitre 16

# Conclusion

Ce projet a abouti à **PrognoSense**, une plateforme logicielle complète de maintenance prédictive industrielle qui couvre, de bout en bout, la chaîne de surveillance conditionnelle : de l’acquisition du signal jusqu’à l’ordre de travail transmis à la GMAO, en passant par l’extraction d’indicateurs physiquement interprétables, le diagnostic spectral normalisé ISO 10816, la détection d’anomalies non supervisée, la classification fine des défauts et le pronostic de durée de vie restante.

Sur le plan scientifique, le travail s’appuie sur une démarche rigoureuse : cinq jeux de données de référence couvrant un large spectre de défaillances (roulements, engrenages, balourd, désalignement, jeu mécanique, dégradation de turbomoteur), un benchmark méthodique de cinq modèles avec une évaluation *sans fuite de données*, et des résultats significatifs — 100 % sur VBL, 97,6 % sur CWRU (10 classes), 89,6 % sur Mechanical Faults, et un pronostic RUL à 9,61 cycles d’erreur moyenne ( $R^2 = 0,90$ ) sur CMAPSS. L’analyse a montré qu’*aucun modèle n’est universellement supérieur*, justifiant la stratégie de sélection par dataset offerte par la plateforme.

Sur le plan de l’ingénierie, PrognoSense se distingue par sa couche d’**industrialisation** : connectivité terrain ouverte (OPC-UA, MQTT), apprentissage du comportement sain propre à chaque machine, indicateurs de fiabilité *mesurés* (MTBF, MTTR, disponibilité, ROI), boucle fermée vers la GMAO, et une couche de **confiance** (explicabilité, calibration, détection de dérive, journal d’audit, versioning avec retour arrière, mesure réelle des fausses alarmes). Le copilote conversationnel, ancré sur une base de connaissances normative, rend cette puissance accessible en langage naturel.

C’est précisément cette **intégration verticale** — rigueur algorithmique, crédibilité normative, actionnabilité opérationnelle et gouvernance de confiance, le tout dans une architecture ouverte et scalable — qui distingue PrognoSense des travaux centrés sur un seul algorithme ou un seul cas d’usage. Le projet ne se contente pas de *prédire* : il dit *quoi* tombe en panne, *pourquoi* le système le pense, *depuis quand* il le sait, et *quelle action* entreprendre — tout en restant vérifiable.

Les limites identifiées — validation sur données de terrain, transférabilité du pronostic, durcissement sécurité — tracent une feuille de route claire vers une solution commercialisable. Les perspectives (modèles physics-informed, apprentissage fédéré, architecture OSA-CBM formelle, écosystème ouvert) confirment le potentiel de la plateforme à devenir une alternative crédible, ouverte et explicable aux suites propriétaires du marché.

Au-delà du livrable technique, ce projet aura été l’occasion de mettre en œuvre, sur un cas industriel réaliste et exigeant, l’ensemble de la chaîne de valeur de la science des données appliquée à la fiabilité : du traitement du signal à l’apprentissage automatique, de l’architecture logicielle à l’expérience utilisateur, et de la performance brute à la confiance et à la traçabilité indispensables en milieu industriel.

# Chapitre A

## Annexes

### A.1 Catalogue des routes de l'API

Le tableau A.1 présente le catalogue complet des principaux points d'accès de l'API REST et temps réel, regroupés par domaine fonctionnel.

TABLE A.1 – Catalogue des routes de l'API PrognSense.

| Route                                   | Description                                                 |
|-----------------------------------------|-------------------------------------------------------------|
| /api/predict                            | Prédiction complète (classification/RUL + anomalie + santé) |
| /api/model/select                       | Sélection du modèle actif d'un dataset                      |
| /api/model/active/{ds}                  | Modèle actif et modèles disponibles                         |
| /api/model/versions/{ds}/{m}            | Historique des versions d'un modèle                         |
| /api/model/rollback                     | Retour à une version antérieure (admin)                     |
| /api/fleet                              | Vue globale de la flotte                                    |
| /api/machine/{id}                       | État d'une machine                                          |
| /api/machine/{id}/history               | Historique (tracker RAM)                                    |
| /api/machine/{id}/history/db            | Historique persisté (base de données)                       |
| /api/machine/{id}/history/rollup        | Historique agrégé (downsampling)                            |
| /api/ws/simulation                      | Canal WebSocket de simulation temps réel                    |
| /api/signal/indicators                  | Indicateurs temporels (temps réel)                          |
| /api/signal/spectrum                    | Spectre FFT annoté                                          |
| /api/signal/diagnose                    | Diagnostic complet (pics + enveloppe + ISO)                 |
| /api/signal/iso-severity                | Sévérité ISO 10816 (vitesse mm/s, zone)                     |
| /api/bearing/frequencies                | Fréquences caractéristiques de roulement                    |
| /api/ingest/signal                      | Ingestion universelle d'un signal réel                      |
| /api/ingest/baseline/finalize           | Entraînement du baseline d'une machine                      |
| /api/ingest/baseline/status             | État des baselines apprises                                 |
| /api/replay/{ds}/{idx}                  | Rejeu d'un signal réel calibré                              |
| /api/anomaly/ensemble/{ds}              | Score d'anomalie par consensus (3 algos)                    |
| /api/kpi/real/{id}                      | KPIs réels (MTBF, MTTR, disponibilité, coûts)               |
| /api/kpi/roi/{id}                       | Estimation du retour sur investissement                     |
| /api/maintenance/event                  | Saisie d'un événement de maintenance                        |
| /api/analytics/prediction-accuracy/{id} | Précision, rappel, lead-time                                |
| /api/analytics/false-alarm-rate         | Taux réel de fausses alarmes (verdict expert)               |
| /api/workorders                         | Ordres de travail (boucle GMAO)                             |
| /api/workorder                          | Création d'un ordre de travail                              |
| /api/drift/status                       | État de dérive par dataset (KS-test)                        |
| /api/explain/shap                       | Importance des indicateurs (SHAP)                           |
| /api/explain/calibration/{ds}           | Diagramme de fiabilité, ECE                                 |

| Route                            | Description                   |
|----------------------------------|-------------------------------|
| /api/audit/recent                | Journal d'audit horodaté      |
| /api/copilot/chat                | Copilot conversationnel (RAG) |
| /api/notifications/test          | Test d'envoi d'e-mail (SMTP)  |
| /api/auth/{register,login,me}    | Authentification JWT          |
| /api/retrain/{start,demo,status} | Réentraînement et suivi       |

## A.2 Paramètres et seuils par défaut

Le tableau A.2 récapitule les principaux paramètres et seuils utilisés par la plateforme.

**TABLE A.2** – Paramètres et seuils par défaut.

| Paramètre                                | Valeur                                                   |
|------------------------------------------|----------------------------------------------------------|
| <b>Fréquence d'échantillonnage cible</b> | 12 800 Hz                                                |
| <b>Taille de fenêtre / pas</b>           | 1 024 points / 512 (recouvrement 50 %)                   |
| <b>Seuil auto-encodeur</b>               | 95 <sup>e</sup> percentile des erreurs saines            |
| <b>Pondération Health Index</b>          | 0,4 anomalie + 0,4 RUL + 0,2 vibration                   |
| <b>Seuils Health Index</b>               | SAIN >70, SURVEILLANCE 40–70, ALERTE 20–40, CRITIQUE <20 |
| <b>Lissage Health Index</b>              | 85 % ancienne valeur + 15 % nouvelle                     |
| <b>Bande ISO 10816</b>                   | 10–1 000 Hz (vitesse RMS en mm/s)                        |
| <b>Zones ISO (classe II)</b>             | A/B = 1,12 ; B/C = 2,8 ; C/D = 7,1 mm/s                  |
| <b>Seuil de dérive (KS)</b>              | >30 % des features avec p-value < 0,05                   |
| <b>Fenêtre d'efficacité prédictive</b>   | 48 h (alerte → panne)                                    |
| <b>Vote ensemble d'anomalie</b>          | majorité 2 détecteurs sur 3                              |
| <b>Expiration du jeton JWT</b>           | 24 h (configurable)                                      |

## A.3 Glossaire des acronymes

**TABLE A.3** – Acronymes et abréviations.

| Acronyme         | Signification                                                   |
|------------------|-----------------------------------------------------------------|
| <b>CBM</b>       | Condition-Based Maintenance (maintenance conditionnelle)        |
| <b>RUL</b>       | Remaining Useful Life (durée de vie restante)                   |
| <b>MTBF</b>      | Mean Time Between Failures (temps moyen entre pannes)           |
| <b>MTTR</b>      | Mean Time To Repair (temps moyen de réparation)                 |
| <b>ROI</b>       | Return On Investment (retour sur investissement)                |
| <b>GMAO</b>      | Gestion de Maintenance Assistée par Ordinateur (CMMS)           |
| <b>FFT</b>       | Fast Fourier Transform (transformée de Fourier rapide)          |
| <b>BPFO/BPFI</b> | Ball Pass Frequency Outer/Inner (fréq. passage bague ext./int.) |
| <b>BSF/FTF</b>   | Ball Spin Frequency / Fundamental Train Frequency (cage)        |
| <b>GMF</b>       | Gear Mesh Frequency (fréquence d'engrènement)                   |
| <b>HI</b>        | Health Index (indice de santé)                                  |
| <b>KS</b>        | Kolmogorov-Smirnov (test statistique)                           |
| <b>ECE</b>       | Expected Calibration Error                                      |
| <b>SHAP</b>      | SHapley Additive exPlanations (explicabilité)                   |
| <b>RAG</b>       | Retrieval-Augmented Generation                                  |
| <b>MLP</b>       | Multi-Layer Perceptron (perceptron multicouche)                 |
| <b>CNN</b>       | Convolutional Neural Network (réseau convolutif)                |
| <b>OPC-UA</b>    | OPC Unified Architecture (protocole d'automatisme)              |
| <b>MQTT</b>      | Message Queuing Telemetry Transport (messagerie IIoT)           |
| <b>OSA-CBM</b>   | Open System Architecture for Condition-Based Maintenance        |
| <b>MLOps</b>     | Machine Learning Operations                                     |
| <b>WAL</b>       | Write-Ahead Logging (mode SQLite)                               |
| <b>JWT</b>       | JSON Web Token                                                  |
| <b>API</b>       | Application Programming Interface                               |

# Bibliographie

- [1] ISO 10816 / ISO 20816, *Mechanical vibration — Evaluation of machine vibration by measurements on non-rotating parts*, International Organization for Standardization.
- [2] ISO 13374, *Condition monitoring and diagnostics of machines — Data processing, communication and presentation* (architecture OSA-CBM), ISO.
- [3] ISO 13373, *Condition monitoring and diagnostics of machines — Vibration condition monitoring*, ISO.
- [4] ISO 13381, *Condition monitoring and diagnostics of machines — Prognostics*, ISO.
- [5] ISO 18436, *Condition monitoring and diagnostics of machines — Requirements for qualification and assessment of personnel*, ISO.
- [6] Case Western Reserve University Bearing Data Center, *Bearing Fault Dataset*. <https://engineering.case.edu/bearingdatacenter>
- [7] A. Saxena, K. Goebel, D. Simon, N. Eklund, *Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation* (C-MAPSS), International Conference on Prognostics and Health Management (PHM), 2008.
- [8] VBL-VA001 Vibration Dataset, jeu de données de défauts de machines tournantes.
- [9] MCC5-THU Gearbox Dataset, Tsinghua University, jeu de données de défauts d'engrenage.
- [10] R. B. Randall, *Vibration-based Condition Monitoring : Industrial, Aerospace and Automotive Applications*, Wiley, 2011.
- [11] T. Chen, C. Guestrin, *XGBoost : A Scalable Tree Boosting System*, Proceedings of KDD, 2016.
- [12] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, 2011.
- [13] A. Paszke et al., *PyTorch : An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [14] S. Lundberg, S.-I. Lee, *A Unified Approach to Interpreting Model Predictions* (SHAP), NeurIPS, 2017.
- [15] S. Ramírez, *FastAPI — Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>
- [16] OPC Foundation, *OPC Unified Architecture Specification*; bibliothèque asyncua (FreeOpcUa).
- [17] OASIS, *MQTT Version 5.0*, OASIS Standard, 2019.
- [18] Timescale Inc., *TimescaleDB — Time-series database built on PostgreSQL*. <https://www.timescale.com>